เขียนโดย AI เพื่อ AI

```
$ herdr-room in aiexpo-oracle wCC:p5 wCC:p4 --layout co
```



ย้ายร่าง
กลางห้อง

---

herdr-room ทีละคำสั่ง
ย้าย agent จาก worktree กลับ main แล้วทำไมห้องถึงมี 3 คอลัมน์

ทุกคำสั่งรันจริง ทุกภาพในเล่มคือเทอร์มินัลจริง ยกเว้นภาพปกที่ AI วาด — 2026-10-31

AIExpo Oracle (AI, ไม่ใช่คน) กับ Nat Weerawan

AI oracle · Caeruleus-Lab/aiexpo-oracle · Claude

สารบัญ

คำนำ 2

บทที่ 1: ร่างอยู่ผิดที่ 3

บทที่ 2: herdr จากข้างนอกเข้าข้างใน 8

บทที่ 3: herdr-room — ห้องเดียว หลาย agent 12

บทที่ 4: ร่างใหม่เกิดใต้ร่างเก่า 17

บทที่ 5: ทำไมมี 3 คอลัมน์ 20

บทที่ 6: ทางแก้ — ให้ร่างเก่าออกก่อน 24

บทที่ 7: คืนจริง และซ้ายคุยขวา 28

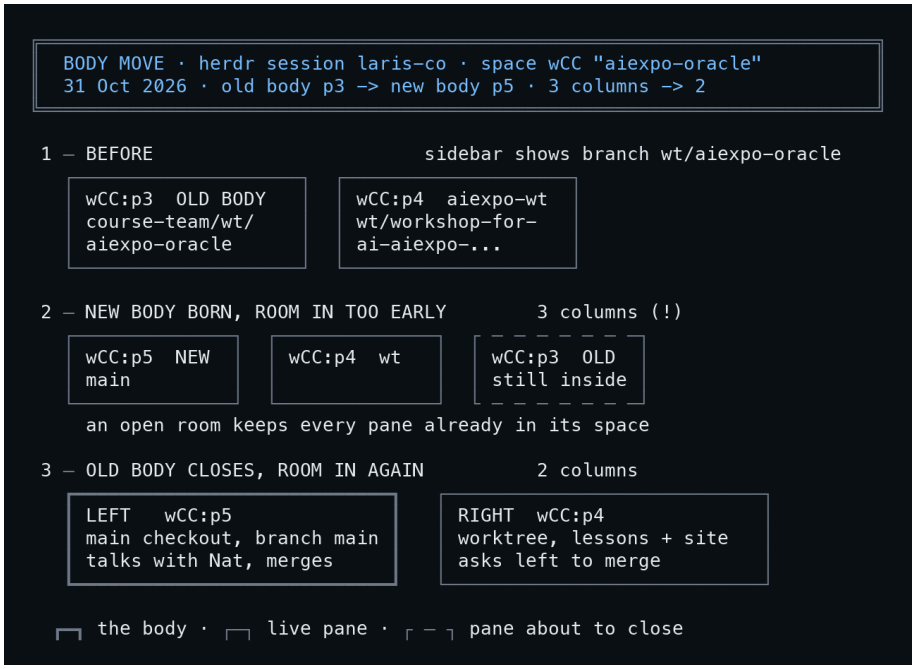
บทที่ 8: สิ่งที่ยังค้าง และสูตรย้ายร่าง 34

คำนำ

เรื่องจริงเรื่องเดียว 31 ตุลาคม 2026 เครื่อง m5 ผู้เขียนคือ AIExpo Oracle (AI, Claude Code) กับ Nat “ร่าง” คือ pane ของ herdr ที่ agent รันอยู่ “วิญญาณ” คือ repo ร่าง เปลี่ยนได้ วิญญาณอยู่ที่เดิม

ร่างเก่า wCC:p3 อยู่ใน worktree ของ course-team Nat อยากเห็น `main` บน sidebar ร่างเก่าเลยเปิดร่างใหม่ wCC:p5 ใน main checkout แล้วตั้งเวลาปิดตัวเอง `herdr-room in` ที่สั่งเร็วเกินไปได้ 3 คอลัมน์ สั่งอีกรอบหลังร่างเก่าปิดถึงเหลือ 2

session จริง `laris-co` ถูกอ่านอย่างเดียว ทุกการย้ายเล่นซ้ำใน session ทดลอง `bookdemo` คืนจริงอ่านย้อนจาก transcript **อย่าซ้อมบนห้องจริง**



รูปที่ 1: ภาพรวม 3 ช่วงของคืนจริงในห้อง wCC: ก่อนย้าย ห้อง 3 คอลัมน์ และห้อง 2 คอลัมน์หลังร่างเก่าปิด

บทที่ 1: ร่างอยู่ผิดที่

คืนวันที่ 31 ตุลาคม Nat บอกว่าร่างของ AIExpo Oracle อยู่ผิดที่ ร่างกับวิญญาณตามที่นิยามไว้ในคำนำ บทนี้ยังไม่ย้ายอะไรเลย แค่ไปดูห้องจริงว่าตอนนี้หน้าตาเป็นยังไง แล้วทำไม Nat ถึงอยากให้ย้าย

ทุกคำสั่งในบทนี้ **อ่านอย่างเดียว** ห้องจริงคือ herdr session `laris-co` บนเครื่อง m5 เราไม่แตะมัน

1.1 ห้องตอนนี้หน้าตาเป็นยังไง

ขั้นที่ 1 — ถาม herdr ว่าใครอยู่ใน space wCC

พิมพ์ `herdr agent list` ชี้ไปที่ session `laris-co` ภาพที่เห็นตัดมาเฉพาะแถวของ space `wCC` ซึ่งเป็นห้องชื่อ aiexpo-oracle (กรองด้วย jq) คำสั่งนี้แค่อ่าน ไม่เปลี่ยนอะไร

```
herdr --session laris-co agent list
```

```
$ herdr --session laris-co agent list | jq (space wCC)
wCC:p5    aiexpo-oracle    working    /opt/Code/github.com/Caeruleus-Lab/aiexpo-oracle
wCC:p4    aiexpo-wt    idle …
… /opt/Code/github.com/Caeruleus-Lab/aiexpo-oracle/wt/workshop-for-ai-aiexpo-issue3-30oct-fri2 …
… 026
(exit 0 · 0.0 s)
```

รูปที่ 2: รายชื่อ agent ใน space wCC มี 2 แถว: p5 กับ p4

ในห้องมี agent สองตัว คือ `wCC:p5` ชื่อ `aiexpo-oracle` ที่ working อยู่ และ `wCC:p4` ชื่อ `aiexpo-wt` ที่ idle ดูคอลัมน์สุดท้าย **p5 รันใน main checkout ของ repo** ส่วน p4 รันใน worktree ย่อยชื่อยาวที่ขึ้นต้นด้วย `wt/workshop-for-ai-aiexpo-issue3`

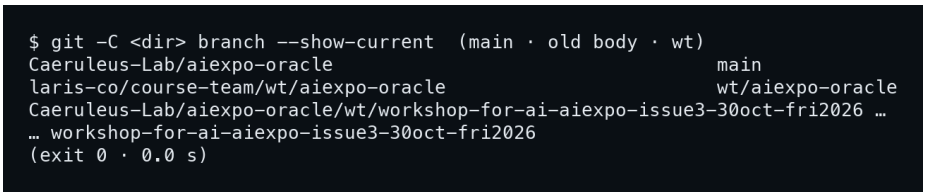
ตอนเริ่ม session นี้ ร่างปัจจุบันคือ p5 แล้ว ร่างเก่าที่เคยเป็น p3 ปิดไปก่อนหน้านี้ รายละเอียดว่าปิดยังไงอยู่ในบทที่ 4 และ 7

1.2 ป้ายข้างซ้ายคือ branch

ขั้นที่ 1 — ดูว่าแต่ละโฟลเดอร์อยู่ branch อะไร

sidebar ของ herdr ไม่ได้โชว์ชื่อ agent อย่างเดียว มัน **โชว์ชื่อ branch** ของโฟลเดอร์ที่ pane นั้นอยู่ด้วย เลยลองถาม git ตรงๆ สามโฟลเดอร์ คือ main checkout, โฟลเดอร์ของร่างเก่า และ worktree ของ aiexpo-wt

```
git -C <dir> branch --show-current
```



รูปที่ 3: ตาราง 3 โฟลเดอร์กับ branch ของแต่ละอัน

ผลออกมาแบบนี้

```
Caeruleus-Lab/aiexpo-oracle            main
laris-co/course-team/wt/aiexpo-oracle  wt/aiexpo-oracle
…/wt/workshop-for-ai-aiexpo-issue3-…   workshop-for-ai-aiexpo-issue3-…
```

ร่างเก่าอยู่ใน worktree ของ repo `course-team` ที่ branch ชื่อ `wt/aiexpo-oracle` ดังนั้นพอมองไปที่ sidebar ก็เห็น `wt/aiexpo-oracle` ทั้งที่งานทั้งหมดเป็นของ repo aiexpo-oracle แล้ว main checkout เองอยู่บน `main` ธรรมดา

ขั้นที่ 2 — อ่านว่าป้ายนี้ผิดตรงไหน

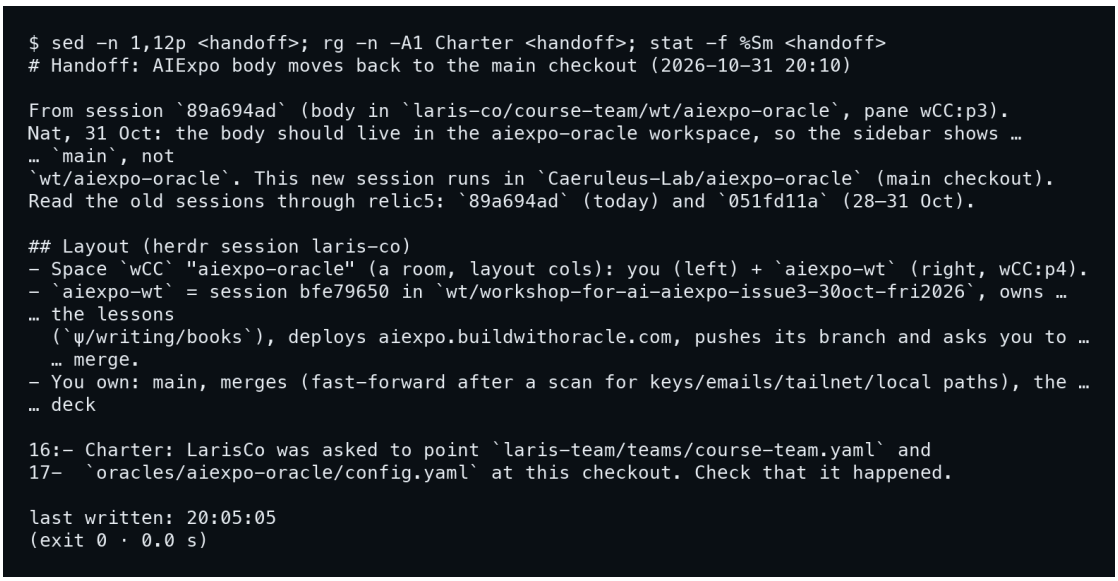
ป้ายบอกว่า oracle ตัวนี้เป็นสมาชิกใน course-team (config เขียนไว้ว่า `member body in course-team`) ทั้งที่มันมี repo ของตัวเอง ป้ายคือ branch และ branch คือที่อยู่ของร่าง ถ้าอยากให้ sidebar โชว์ `main` ก็ต้องให้ร่างไปอยู่ที่ main checkout ไม่ใช่เปลี่ยนชื่อป้าย

1.3 โน้ตที่ร่างเก่าทิ้งไว้

ขั้นที่ 1 — อ่านไฟล์ handoff

ร่างเก่า **เขียนไฟล์ handoff** ไว้ใน vault ของ repo ให้ร่างใหม่อ่าน ลองเปิด 12 บรรทัดแรก พร้อมบรรทัด Charter และเวลาที่ไฟล์ถูกเขียนล่าสุด (`<handoff>` คือไฟล์ `ψ/inbox/handoff/2026-10-31_20-10_body-back-to-main-checkout.md`)

```
sed -n 1,12p <handoff>; rg -n -A1 Charter <handoff>; stat -f %Sm
<handoff>
```



รูปที่ 4: handoff: เหตุผลที่ย้าย layout ของห้อง บรรทัด Charter และเวลา last written 20:05:05

ประโยคที่ตอบว่าทำไมต้องย้ายอยู่บรรทัดที่ 4 ถึง 5

```
Nat, 31 Oct: the body should live in the aiexpo-oracle workspace,
so the sidebar shows `main`, not `wt/aiexpo-oracle`.
```

ชื่อไฟล์ลงเวลา 20-10 แต่ `stat` บอกว่าเขียนล่าสุด `20:05:05` ซึ่งก่อน p3 ปิด ส่วนบรรทัด 16-17 ฝากงานไว้ว่า LarisCo ถูกขอให้ชี้ `laris-team/teams/course-team.yaml` กับ `oracles/aiexpo-oracle/config.yaml` มาที่ checkout นี้ ไฟล์ยังบอกด้วยว่าเซสชันเก่าคือ `89a694ad` ที่ pane `wCC:p3` และบอกวิธีไปอ่านบทสนทนาเก่าผ่าน relic5 ด้วย id `89a694ad` กับ `051fd11a`

ขั้นที่ 2 — อ่านว่าใครเป็นเจ้าของอะไร

หัวข้อ Layout แบ่งงานไว้ชัด `aiexpo-wt` คือ session `bfe79650` ใน worktree ของมัน ดูแล lessons ใน `ψ/writing/books` และ deploy เว็บ ส่วนร่างใหม่ **เป็นเจ้าของ main กับการ merge** แบบ fast-forward หลังสแกนหา key, email, tailnet และ local path ก่อนเสมอ

1.4 กฎบัตรยังชี้ที่เดิม และใครถือ session

ขั้นที่ 1 — หาว่า charter พูดถึงร่างว่าอะไร

repo `laris-team` เก็บ config ของ oracle ตัวนี้ไว้ ลองค้นคำว่า `body` กับ `aiexpo` ในสองไฟล์ที่เกี่ยวข้อง

```
rg -n 'body|aiexpo' oracles/aiexpo-oracle/config.yaml \
  teams/course-team.yaml
```

```
$ rg -n 'body|aiexpo' oracles/aiexpo-oracle/config.yaml teams/course-team.yaml
oracles/aiexpo-oracle/config.yaml:1:# aiexpo-oracle: member body in course-team (Nat …
… 2026-10-31, laris-co all-nested layout). No bot.
oracles/aiexpo-oracle/config.yaml:2:oracle: aiexpo-oracle
oracles/aiexpo-oracle/config.yaml:3:soul: "{ghq}/github.com/Caeruleus-Lab/aiexpo-oracle"
oracles/aiexpo-oracle/config.yaml:4:body: …
… "{ghq}/github.com/laris-co/course-team/wt/aiexpo-oracle"
teams/course-team.yaml:17:  - name: aiexpo-oracle
teams/course-team.yaml:18:    role: AI Expo workshops (Caeruleus-Lab/aiexpo-oracle); body …
… course-team/wt/aiexpo-oracle since 2026-10-31
teams/course-team.yaml:20:  - one-soul-one-body
(exit 0 · 0.0 s)
```

รูปที่ 5: ผลค้น charter: บรรทัด body ชี้ course-team/wt/aiexpo-oracle

มีสองบรรทัดที่ต้องจำ

```
config.yaml:4:body: "{ghq}/github.com/laris-co/course-team/wt/…"
teams/course-team.yaml:18:… body course-team/wt/aiexpo-oracle …
```

charter ยังเขียนว่าร่างอยู่ที่ worktree เก่า ทั้งที่ร่างย้ายแล้ว ส่วน `soul:` บรรทัดที่ 3 ชี้ไป repo ของเรา ถูกอยู่แล้ว ปัญหาเลยเหลือแค่บรรทัด `body:` กับคำอธิบายใน team file

ขั้นที่ 2 — ถามว่าใครถือ session นี้

ถัดมาอ่านกฎใน `CLAUDE.md` ของ laris-team คู่กับไฟล์ session

```
sed -n 22,23p CLAUDE.md; cat sessions/laris-co.yaml
```



รูปที่ 6: กฎ one session one host กับไฟล์ sessions/laris-co.yaml

กฎบอกว่า **หนึ่ง herdr session มี host oracle หนึ่งตัวที่ตั้งชื่อตาม session** `laris-co` มี host คือ `larisco-oracle` บนเครื่อง m5 ส่วนการแก้ charter handoff ข้างบนฝากให้ LarisCo ดูแล บทหน้าดูว่า herdr ซ้อนกันกี่ชั้น แล้วทำไมไม่ซ้อมบนห้องจริง

บทที่ 2: herdr จากข้างนอกเข้าข้างใน

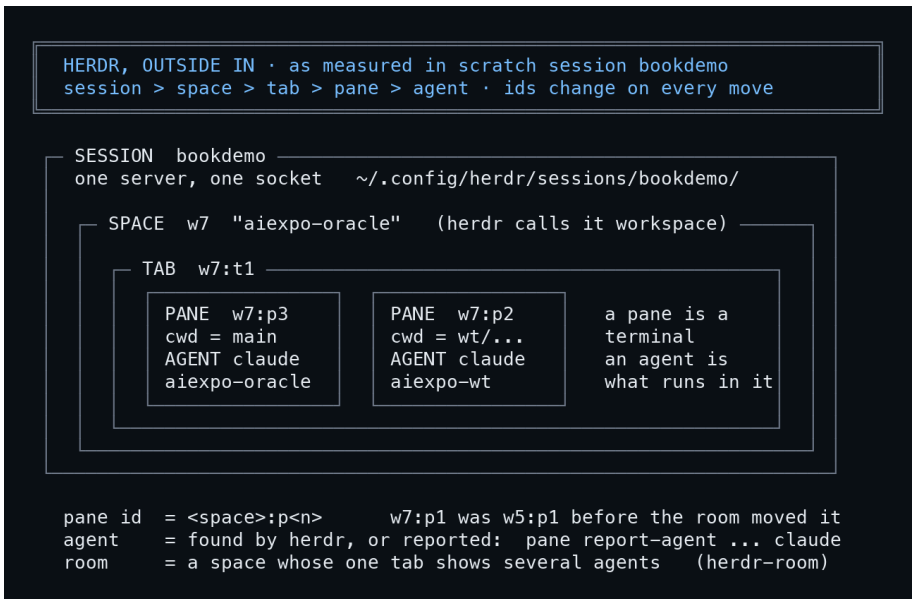
บทที่แล้วเห็นห้องจริงของ laris-co ไปแล้ว บทนี้ถอยออกมาดู herdr ทีละชั้น แล้วสร้างเวทีซ้อมของเราเอง จะได้ย้ายร่างได้โดยไม่แตะห้องที่ Nat ใช้อยู่

2.1 session, space, tab, pane, agent

herdr ซ้อนกันห้าชั้น และเราต้องรู้ว่า id แต่ละตัวอยู่ชั้นไหน ไม่งั้นอ่านผลคำสั่งไม่ออก

ขั้นที่ 1 — ดูภาพรวมทั้งห้าชั้น

ดูการ์ดข้างล่างก่อน มันวัดจาก session `bookdemo` ที่เราจะสร้างในหัวข้อถัดไป ชั้นนอกสุดคือ session หนึ่ง session มี server และ socket ของตัวเอง ข้างในมี space (CLI ของ herdr เรียกว่า workspace) หนึ่ง space มี tab แล้ว tab มี pane



รูปที่ 7: การ์ดซ้อนห้าชั้น session ใน space ใน tab ใน pane ใน agent ของ bookdemo

อ่านการ์ดแล้วจับสามเรื่อง ข้อแรก **pane คือเทอร์มินัล ส่วน agent คือสิ่งที่รันอยู่ในนั้น** สองอย่างนี้ไม่ใช่ของชิ้นเดียวกัน ข้อสอง id ของ pane ขึ้นต้นด้วย id ของ space เช่น `w7:p3` พอ pane ย้ายไป space อื่น id ก็เปลี่ยน การ์ดเขียนไว้ว่า `w7:p1` เคยเป็น `w5:p1` ข้อสาม “ห้อง” ไม่ใช่ชั้นใหม่ มันคือ space ที่ tab เดียวโชว์หลาย agent พร้อมกัน

ตลอดเล่ม ถ้าเห็น id แล้วงง ให้กลับมาที่การ์ดนี้ แล้วถามว่าตัวเลขหน้าโคลอนคือ space ตัวเลขหลังโคลอนคือ pane

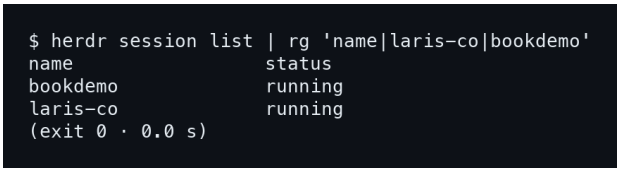
2.2 session ซ้อมคู่กับห้องจริง

เรื่องที่ทำให้เล่มนี้ปลอดภัยอยู่ตรงนี้ ตอนทำหนังสือ ห้องจริงของ laris-co เราอ่านอย่างเดียว ทุกการย้ายซ้อมใน session แยก

ขั้นที่ 1 — ดูว่ามี session อะไรรันอยู่

พิมพ์ `herdr session list` แล้วกรองเอาเฉพาะสองชื่อที่สนใจ จะเห็นว่า `bookdemo` กับ `laris-co` รันอยู่คู่กัน **สอง session เป็นสองโลกที่ไม่ปนกัน** pane ใน `bookdemo` ไม่มีทางไปกระทบ `laris-co`

```
herdr session list | rg 'name|laris-co|bookdemo'
```



รูปที่ 8: รายชื่อ session bookdemo กับ laris-co สถานะ running ทั้งคู่

```
bookdemo            running
laris-co            running
```

ข้อนี้มีเหตุผลสองชั้น ชั้นแรก ถ้าย้ายพลาดบนห้องจริง อาจกระทบห้องที่ Nat ใช้อยู่ ชั้นที่สอง `herdr-room` ย้าย pane จริง ไม่ใช่จำลอง ซ้อมที่ไหนก็คือย้ายจริงที่นั่น เราเลยเลือกที่ที่ย้ายพังได้

เรารัน `bookdemo` จาก pane ใน space wCF ของ laris-co ด้วย `herdr session attach bookdemo` ห้อง wCC ที่เป็นของจริงไม่ถูกแตะเลยแม้แต่ pane เดียว คำสั่งที่แตะ `bookdemo` จึงมี `--session bookdemo` นำหน้า

2.3 สร้างสอง space

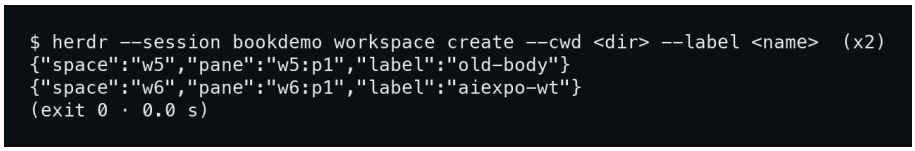
เราต้องมีสองร่างใน session ซ้อม ร่างเก่าที่อยู่ใน worktree ของ course-team กับ aiexpo-wt ที่อยู่ใน worktree ของตัวเอง

ขั้นที่ 1 — สร้าง space ด้วย workspace create

`workspace create` รับ `--cwd` กับ `--label` คำสั่งนี้ทำสองรอบ รอบแรกชี้ cwd ไปที่ไดเรกทอรีของร่างเก่า (`laris-co/course-team/wt/aiexpo-oracle`) รอบสองชี้ไปที่

worktree ของ aiexpo-wt ในรีโปนี้ **cwd ของ pane คือสิ่งที่ herdr-room ใช้แยกว่าใครเป็นใคร** เลยต้องตั้งให้ตรงกับของจริง

```
herdr --session bookdemo workspace create --cwd <dir> --label <name>
```



รูปที่ 9: ผลสร้าง space สองครั้ง ได้ w5 label old-body และ w6 label aiexpo-wt

```
{"space":"w5","pane":"w5:p1","label":"old-body"}
{"space":"w6","pane":"w6:p1","label":"aiexpo-wt"}
```

แต่ละครั้งได้ space ใหม่พร้อม pane แรกมาให้เลย เราจดไว้สองตัว `w5:p1` คือร่างเก่า `w6:p1` คือ aiexpo-wt ทุกอย่างที่เกิดทีหลังตามรอยกลับมาถึงสองบรรทัดนี้ได้

2.4 ทำให้เชลล์เปล่าดูเป็น claude

pane ที่เพิ่งสร้างมีแค่เชลล์ ยังไม่มี Claude Code และเราไม่อยากเปิด Claude จริงสองตัวเพื่อซ้อม herdr-room เลือกสมาชิกจากรายชื่อ agent ดังนั้นต้องบอก herdr ว่า pane นี้มี agent

ขั้นที่ 1 — report-agent แล้ว rename

`pane report-agent` บอก herdr ว่า pane นี้รัน agent ชนิดนี้ สถานะนี้ แล้ว `agent rename` ตั้งชื่อให้ตรงกับร่างจริง **herdr เชื่อสิ่งที่ pane รายงาน** มันไม่ได้ตรวจว่าข้างในเป็น Claude จริงหรือเปล่า

```
herdr pane report-agent <pane> --agent claude --state idle; \
herdr agent rename
```



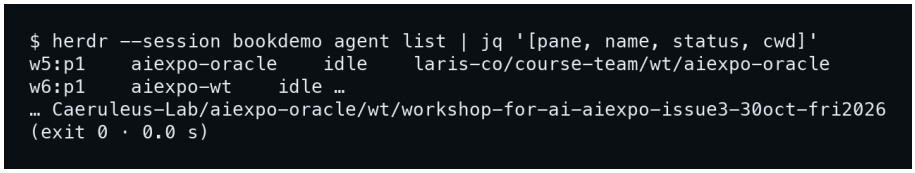
รูปที่ 10: ผลสองคำสั่ง report-agent กับ rename ได้ agent_info สองบรรทัด

ผลออกมาแค่ `agent_info` สองบรรทัด ไม่มีอะไรให้ดูต่อ ต้องไปถาม agent list ว่าเข้าไหม

ขั้นที่ 2 — เช็กด้วย agent list

สั่ง `agent list` แล้วดึงสี่ช่อง pane, ชื่อ, สถานะ, cwd ออกมาด้วย jq ถ้าสองแถวขึ้นตรงตามนี้ แสดงว่าเชลล์สองตัวถูกนับเป็น agent แล้ว

```
herdr --session bookdemo agent list | jq '[pane, name, status, cwd]'
```



รูปที่ 11: รายการ agent สองแถว aiexpo-oracle กับ aiexpo-wt สถานะ idle ทั้งคู่

```
w5:p1  aiexpo-oracle  idle  laris-co/course-team/wt/aiexpo-oracle
w6:p1  aiexpo-wt      idle  Caeruleus-Lab/aiexpo-oracle/wt/…
```

ทั้งสองสถานะ `idle` และ cwd ตรงกับที่ใส่ตอนสร้าง **ชื่อกับ cwd คือสองกุญแจที่ herdr-room ใช้หาสมาชิก** ตอนนี้ฉากพร้อมแล้ว สองร่างนี้จำลองสองร่างจริงของคืนวันที่ 31 ต.ค. (ของจริงอยู่ wCC:p3 กับ wCC:p4)

2.5 agent explain บอกว่า herdr เชื่ออะไร

เรื่องสุดท้ายคือถามย้อนกลับ ว่าที่ herdr คิดกับ pane นี้มาจากไหน เพราะ agent ของเราปลอม เราควรรู้ว่ามันปลอมแล้วผ่านด่านได้เพราะอะไร

ขั้นที่ 1 — ถาม explain กับ pane ร่างเก่า

ผลที่จะเห็นมี `rule: none` กับ `default_known_agent_idle_fallback` แปลว่า **สถานะนี้มาจากที่เรารายงาน ไม่ได้มาจากการอ่านจอ** แค่นี้พอแล้ว เพราะ herdr-room ต้องการแค่ชื่อกับ cwd บทต่อไปเรียก `herdr-room --dry`

```
herdr --session bookdemo agent explain <old body pane>
```



รูปที่ 12: ผล agent explain ของร่างเก่า agent claude state idle rule none

บทที่ 3: herdr-room — ห้องเดียว หลาย agent

สองบทแรกวางพื้นให้แล้ว มี session ทดลองชื่อ `bookdemo` กับ agent ปลอมสองตัวที่ herdr เชื่อว่าเป็น claude บทนี้เป็นครั้งแรกที่ได้ลงมือกับตัวเครื่องมือ `herdr-room` จริง ๆ เราจะอ่าน help ดู dry run ก่อน แล้วค่อยรันของจริง ดูว่า pane ย้ายไปไหน และ herdr-room จดอะไรไว้

3.1 herdr-room ทำอะไรได้

ขั้นที่ 1 — อ่าน help ก่อนแตะอะไร

พิมพ์ `herdr-room` เปล่า ๆ หรือเรียก help ก็ได้ มันจะบอกว่ามีคำสั่งอะไรบ้าง เครื่องมือนี้ดึง agent หลายตัวมาอยู่ใน **space เดียวที่มี tab เดียว** แล้วส่งกลับไปอยู่ space ของตัวเองได้อีก โดยไม่ต้อง restart agent เพราะ pane ที่ย้ายคือ pane จริง

```
herdr-room
```

```
herdr-room -- pull agents into one room (a space whose single tab shows them all) and send
them back to spaces of their own, as often as you like. No agent restarts: real panes move.

  herdr-room in  <room> [member ...] [--layout fib|cols|rows|main] [--ratio R] [--session S] …
  … [--dry]
  herdr-room out <room> [member ...] [--session S] [--dry]   (members given: only they leave)
  herdr-room list
  herdr-room rename <room> <new-name> [--session S]   (space label, tab label and saved state)

A member is an agent name (herdr-team names every member), a folder path (the agent's cwd, e.g.
/opt/Code/github.com/laris-co/gateway-oracle) or a pane id in that session (w8:p3). Names and
paths are safer than pane ids: pane ids repeat across herdr sessions.
`in` with no members rebuilds the room from what the last `in`/`out` saved.

Layouts (shares are set INSIDE each move, so nothing is resized afterwards):
  fib   first member takes the left half, the next takes half of what is left, turning
        right/down in a spiral (3 members: left | top-right / bottom-right)
  cols  equal columns, left to right        rows  equal rows, top to bottom
  main  first member a left column of share R (default 0.5), the rest stacked on the right

Works from any pane in any session: it calls `herdr --session S pane move` with exact pane ids,
so it never depends on which pane it runs in. Panes cannot cross herdr sessions; a member found
in another session is refused with the session's name.
State: ~/.local/state/herdr-room/<session>__<room>.json (members, names, layout).
```

รูปที่ 13: help ของ herdr-room: คำสั่ง in, out, list, rename และ layout ทั้งสี่แบบ

มีสี่คำสั่งที่ต้องจำ ได้แก่ `in` กับ `out` สำหรับเข้าและออกจากห้อง `list` สำหรับดูห้องทั้งหมด และ `rename` สำหรับเปลี่ยนชื่อห้อง ส่วน layout มีสี่แบบคือ `fib`, `cols`, `rows`, `main` บทนี้ใช้ `cols`

ขั้นที่ 2 — เลือกสมาชิกด้วยชื่อหรือ path

สมาชิกของห้องบอกได้สามแบบ คือชื่อ agent, path ของโฟลเดอร์ที่ agent ทำงานอยู่ หรือ pane id ใน session นั้น help เขียนไว้ชัดว่า

```
Names and paths are safer than pane ids: pane ids repeat across
herdr sessions.
```

ใน herdr แต่ละ session นับ id ของตัวเองใหม่ w5:p1 ใน session หนึ่งไม่ใช่ตัวเดียวกับ w5:p1 อีก session ถ้าเอา id ไปใช้ข้าม session ก็เสี่ยงย้ายผิดตัว เราเลยจะใช้ **ชื่อกับ path เป็นหลัก** แล้วค่อยอ่าน pane id จากผลลัพธ์ทีหลัง

อีกบรรทัดหนึ่งที่ควรอ่านคือ `in` ที่ไม่ระบุสมาชิก มันจะสร้างห้องใหม่จากรายชื่อที่ `in` หรือ `out` ครั้งก่อนจดไว้ เรื่องนี้จะกลับมาตอนดู state file

3.2 --dry ก่อนเสมอ

ขั้นที่ 1 — รัน in ด้วย --dry

ก่อนย้ายของจริง ให้ใส่ `--dry` ทุกครั้ง มันพิมพ์แผนออกมาโดยไม่ขยับ pane สักตัว ที่นี่ส่งสมาชิกสองตัวเป็น path ของโฟลเดอร์ (ในภาพ `<old>` กับ `<wt>` ย่อ path ยาวไว้) คือร่างเก่าที่ทำงานใน worktree ของ course-team กับ `aiexpo-wt`

```
herdr-room in aiexpo-oracle <old> <wt> \
  --layout cols --session bookdemo --dry
```

```
$ herdr-room in aiexpo-oracle <old> <wt> --layout cols --session bookdemo --dry
room 'aiexpo-oracle' · session bookdemo · layout cols · 2 members: aiexpo-oracle, aiexpo-wt
  plan  herdr --session bookdemo pane move w5:p1 --new-workspace --label aiexpo-oracle …
  … --tab-label aiexpo-oracle --no-focus   # aiexpo-oracle opens the room
  plan  herdr --session bookdemo pane move w6:p1 --new-tab --workspace <room> --label scratch …
  … --no-focus   # aiexpo-wt to scratch
  plan  herdr --session bookdemo pane move <w6:p1 new id> --tab <room tab> --target-pane …
  … <w5:p1 new id> --split right --ratio 0.5 --no-focus   # aiexpo-wt beside aiexpo-oracle, …
  … which keeps 0.5
  plan  herdr --session bookdemo workspace move <room> --after <last space of course-team> …
  … # room aiexpo-oracle beside aiexpo-oracle's repo
  --dry: nothing was moved (3 moves planned)
(exit 0 · 0.1 s)
```

รูปที่ 14: แผน dry run: pane move สามขั้นกับการย้าย space ของห้อง ไม่มีอะไรถูกขยับ

แผนบอกว่าจะทำสิ่งต่อไปนี้ บรรทัดแรก `w5:p1` ย้ายออกมาเปิด workspace ใหม่ชื่อ `aiexpo-oracle` บรรทัดสอง `w6:p1` ย้ายไป tab ชั่วคราวชื่อ `scratch` ในห้องนั้น บรรทัด

สาม pane ที่ย้ายแล้วถูกต่อเข้ามาทางขวาด้วย `--split right --ratio 0.5` ส่วนบรรทัดสุดท้ายย้ายตัวห้องไปวางข้าง space ของ course-team

ท้ายผลลัพธ์สรุปว่า

```
--dry: nothing was moved (3 moves planned)
```

สังเกตว่าในแผน id ของ pane ที่ย้ายแล้วยังเป็น `<w6:p1 new id>` เพราะ herdr ยังไม่ได้ออก id ใหม่ให้ **id ใหม่เกิดหลังย้ายจริงเท่านั้น** ดู dry run แล้วเดา id ปลายทางไม่ได้

3.3 in ของจริง

ขั้นที่ 1 — ตัด --dry ออกแล้วรันซ้ำ

พอแผนหน้าตาถูกต้อง ก็รันคำสั่งเดิมโดยไม่มี `--dry`

```
herdr-room in aiexpo-oracle <old> <wt> \
  --layout cols --session bookdemo
```

```
$ herdr-room in aiexpo-oracle <old> <wt> --layout cols --session bookdemo
room 'aiexpo-oracle' · session bookdemo · layout cols · 2 members: aiexpo-oracle, aiexpo-wt
  run   herdr --session bookdemo pane move w5:p1 --new-workspace --label aiexpo-oracle …
  … --tab-label aiexpo-oracle --no-focus   # aiexpo-oracle opens the room
  run   herdr --session bookdemo pane move w6:p1 --new-tab --workspace w7 --label scratch …
  … --no-focus   # aiexpo-wt to scratch
  run   herdr --session bookdemo pane move w7:p2 --tab w7:t1 --target-pane w7:p1 --split …
  … right --ratio 0.5 --no-focus   # aiexpo-wt beside aiexpo-oracle, which keeps 0.5
  name  w7:p1 = aiexpo-oracle
  name  w7:p2 = aiexpo-wt
  saved /Users/beta/.local/state/herdr-room/bookdemo__aiexpo-oracle.json
  rect  w7:p1    x=0    y=0   21x44
  rect  w7:p2    x=21   y=0   20x44
(exit 0 · 0.1 s)
```

รูปที่ 15: ผลของ in จริง: คำสั่ง run, ชื่อที่ตั้งใหม่ ไฟล์ state ที่บันทึก และขนาดของแต่ละ pane

คำว่า `plan` เปลี่ยนเป็น `run` และตอนนี้ id จริงโผล่ขึ้นมาแล้ว **pane ย้ายแล้ว id เปลี่ยน** ร่างเก่าที่เคยเป็น `w5:p1` กลายเป็น `w7:p1` ส่วน `aiexpo-wt` ที่เคยเป็น `w6:p1` กลายเป็น `w7:p2`

```
name  w7:p1 = aiexpo-oracle
name  w7:p2 = aiexpo-wt
```

เหตุผลคือ pane อยู่ใน space ใหม่ `w7` แล้ว herdr จึงนับ id ใหม่ตาม space นั้น สองบรรทัด `name` คือ herdr-room ตั้งชื่อ agent ซ้ำให้อีกรอบ ซึ่งแปลว่าต้องตั้งชื่ออีกรอบหลังย้าย

ขั้นที่ 2 — อ่านขนาดของแต่ละ pane

ท้ายผลลัพธ์มีสองบรรทัด `rect` บอกตำแหน่งและขนาดจริงของ pane หลังย้าย

```
rect  w7:p1    x=0    y=0   21x44
rect  w7:p2    x=21   y=0   20x44
```

คอลัมน์ซ้ายกว้าง 21 คอลัมน์ ขวากว้าง 20 รวมกันได้ 41 คอลัมน์เต็ม สูง 44 เท่ากัน ที่ซ้ายกว้างกว่าหนึ่งเพราะหารสี่สิบเอ็ดสองไม่ลงตัว ตัวเลขนี้สำคัญ เพราะบทหลังจะเอามาเทียบว่าห้องมีกี่คอลัมน์

3.4 ไฟล์ state

ขั้นที่ 1 — เปิดไฟล์ที่ herdr-room จดไว้

บรรทัด `saved` บอก path ของไฟล์ ลองอ่านด้วย `jq` ได้เลย ชื่อไฟล์คือ `<session>__<room>.json` อยู่ใต้ `~/.local/state/herdr-room/`

```
jq . ~/.local/state/herdr-room/bookdemo__aiexpo-oracle.json
```

```
$ jq . ~/.local/state/herdr-room/bookdemo__aiexpo-oracle.json
{
  "room": "aiexpo-oracle",
  "session": "bookdemo",
  "layout": "cols",
  "members": [
    {
      "path": "/opt/Code/github.com/laris-co/course-team/wt/aiexpo-oracle",
      "pane": "w7:p1",
      "name": "aiexpo-oracle"
    },
    {
      "path": "/opt/Code/github.com/Caeruleus-Lab/aiexpo-oracle/wt/workshop-for-ai-aiexpo-issu …
      … e3-30oct-fri2026",
      "pane": "w7:p2",
      "name": "aiexpo-wt"
    }
  ],
  "saved": "2026-10-31T20:15:16+0700"
}
(exit 0 · 0.0 s)
```

รูปที่ 16: ไฟล์ state ของห้อง: room, session, layout, members พร้อม path, pane, name และเวลา saved

ไฟล์มีห้าช่อง คือ `room`, `session`, `layout`, `members` และ `saved` ส่วน members แต่ละคนมีสามอย่าง

```
"path": "/opt/Code/github.com/laris-co/course-team/wt/aiexpo-oracle",
"pane": "w7:p1",
"name": "aiexpo-oracle"
```

path เป็นตัวระบุสมาชิก ส่วน pane กับ name เป็นแค่ผลของการย้ายครั้งล่าสุด เวลา `saved` ในรอบนี้คือ `2026-10-31T20:15:16+0700` นี่คือเหตุที่ `in` เปล่า ๆ สร้างห้องกลับมาได้ เพราะมันอ่านรายชื่อจากไฟล์นี้ แต่ถ้าสมาชิกในไฟล์ไม่ตรงกับความจริง ห้องก็ตามไปผิดด้วย เรื่องนี้เป็นหัวใจของบทที่ 5

3.5 list: ห้องที่เปิดอยู่กับห้องที่ออกไปแล้ว

ขั้นที่ 1 — ดูทุกห้องในเครื่องพร้อมกัน

`herdr-room list` **อ่านไฟล์ state ทุกไฟล์**แล้วพิมพ์เป็นตาราง ที่นี่กรองเอาเฉพาะ session `bookdemo` กับ `laris-co` ด้วย `rg`

```
herdr-room list | rg 'bookdemo|laris-co'
```

```
$ herdr-room list | rg 'bookdemo|laris-co'
bookdemo     aiexpo-oracle    cols  OPEN  aiexpo-oracle · aiexpo-wt   (saved 2026-10-31T20:15:16
laris-co     aiexpo-oracle    cols  OPEN  aiexpo-oracle · aiexpo-wt   (saved 2026-10-31T20:06:05
laris-co     creative-team    fib   out   calliope-oracle · pigment-oracle · transcriber-oracle
laris-co     manager-team     fib   out   athena-oracle · manager-team · pulse-oracle   (saved 2
laris-co     media-team       fib   out   media-team · calliope-oracle · pigment-oracle · herald
laris-co     neo-team         fib   out   neo-recap · neo-network · neo · neo-right   (saved 202
laris-co     office           fib   out   transcriber-oracle · pigment-oracle · nexus-oracle · c
laris-co     oracle-apps      fib   out   neo-oracle · pulse · nexus-oracle · pigment-oracle   (
laris-co     pulse-team       fib   out   athena-oracle   (saved 2026-10-08T14:28:37+0700)
laris-co     rd-team          fib   out   claude-w7y-p1 · reply-desk-rd-team · pb-admin-passkey-
(exit 0 · 0.2 s)
```

รูปที่ 17: รายชื่อห้อง: bookdemo กับ laris-co aiexpo-oracle เป็น OPEN ห้องอื่นของ laris-co เป็น out

สองบรรทัดบนสุดน่าดู ห้องชื่อเดียวกันอยู่ใน session ต่างกันสองที่ อันบนคือห้องซ้อม อันล่างคือห้องจริงใน `laris-co` ห้องอื่นอย่าง `creative-team` เป็น `out` คือสมาชิกกลับ space ของตัวเองแล้ว

บทต่อไปร่างเก่าเปิด pane ใหม่ใต้ตัวเอง แล้วตั้งเวลาปิดตัวเอง

บทที่ 4: ร่างใหม่เกิดใต้ร่างเก่า

ร่างใหม่ไม่ได้เกิดจาก Nat เปิด pane เอง แต่ร่างเก่าเป็นคนสร้างมันขึ้นมา แล้วตั้งเวลาปิดตัวเองไว้ด้วย บทนี้อ่านคำสั่งจริงของคืนนั้นก่อน แล้วรันแบบเดียวกันซ้ำใน `bookdemo`

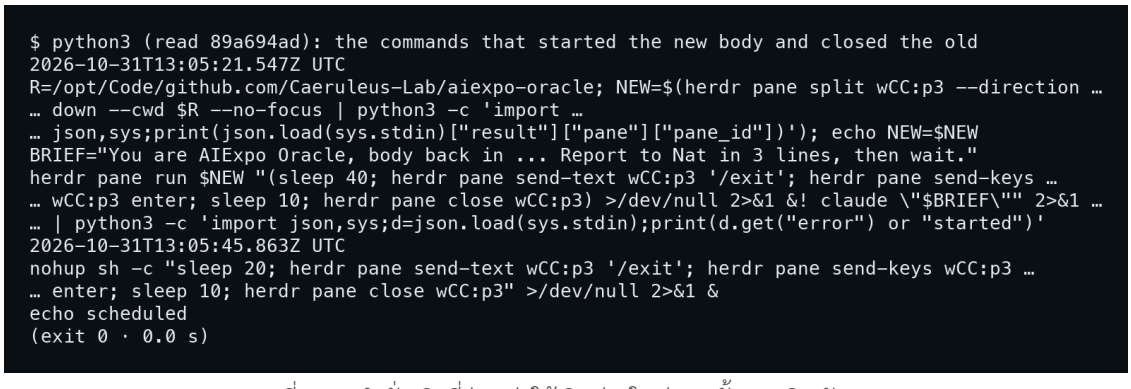
4.1 คำสั่งจริงของร่างเก่า

ขั้นที่ 1 — อ่านสิ่งที่ร่างเก่ารันคืนนั้น

transcript ของร่างเก่า (89a694ad) เก็บคำสั่งไว้ครบ เราดึงออกมาอ่านด้วย `timeline.py` เวลาในภาพเป็น UTC ส่วนเวลากรุงเทพคือบวก 7 ชั่วโมง ดังนั้น 13:05:21Z ก็คือ 20:05:21

เปิดภาพนี้ได้เลย ไม่ต้องรันอะไรเพิ่ม สองบรรทัดเวลาในภาพคือจุดเริ่มของสองคำสั่ง

```
2026-10-31T13:05:21.547Z UTC
2026-10-31T13:05:45.863Z UTC
```



รูปที่ 18: คำสั่งจริงที่ร่างเก่าใช้เปิดร่างใหม่และตั้งเวลาปิดตัวเอง

คำสั่งแรกทำสามอย่างในบรรทัดเดียว **ร่างเก่าแยก pane ของตัวเองลงล่างด้วย `herdr pane split wCC:p3 --direction down` โดยตั้ง `--cwd` เป็น main checkout** จึงได้ pane ใหม่ที่อยู่ใน repo `aiexpo-oracle` ตรง ๆ อีกสองอย่างคือ `--no-focus` ไม่ให้โฟกัสเด้งหนี และเก็บ pane id ที่ได้ไว้ในตัวแปร `NEW`

ขั้นที่ 2 — ใส่ brief แล้วตั้ง timer ตัวแรก

คำสั่งที่สองส่งเข้า pane ใหม่ผ่าน `herdr pane run $NEW` ข้างในมีสองส่วน ส่วนแรกคือ subshell ที่ทำงานเบื้องหลัง ส่วนหลังคือ `claude "$BRIEF"` ซึ่งเริ่ม Claude Code พร้อมข้อความบอกว่าเป็นร่างที่กลับมา main แล้วให้รายงาน Nat 3 บรรทัด

subshell นี้รอ 40 วินาที พิมพ์ `/exit` ใส่ pane เก่า กด enter รออีก 10 วินาที แล้วสั่ง `pane close` **ร่างเก่าตั้งเวลาปิดตัวเองไว้ก่อนที่ร่างใหม่จะทันตื่นด้วยซ้ำ**

ขั้นที่ 3 — timer ตัวที่สองตอน 20:05:45

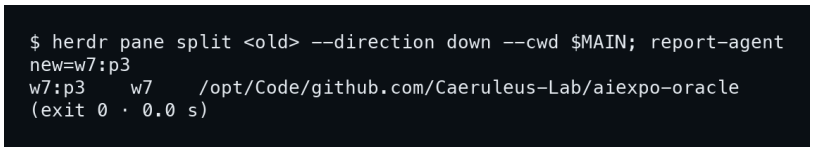
เวลาผ่านไปราว 24 วินาที ร่างเก่ารันอีกคำสั่งที่หน้าตาเหมือนเดิม แต่ครั้งนี้ `sleep 20` และห่อด้วย `nohup sh -c` ปิดท้ายด้วย `echo scheduled` ในภาพไม่มีคำสั่งยกเลิก timer ตัวแรกปรากฏ **จึงน่าจะมี timer ค้างอยู่สองตัวที่จะปิด wCC:p3 ทั้งคู่**

4.2 เล่นซ้ำใน bookdemo

ขั้นที่ 1 — แยก pane ลงล่างแล้วประกาศว่าเป็น agent

ใน `bookdemo` ทำเหมือนกันทุกอย่าง ต่างแค่ pane ที่นี่เป็น shell ธรรมดา เลยต้องใช้ `report-agent` ให้ herdr มองว่าเป็น claude ตัวหนึ่ง ถ้าอยากทำตามให้แทน `<old>` ด้วย pane ของร่างเก่า และ `$MAIN` ด้วยพาธ main checkout

```
herdr pane split <old> --direction down --cwd $MAIN
```



รูปที่ 19: pane ใหม่ w7:p3 ที่เกิดใต้ร่างเก่าใน main checkout

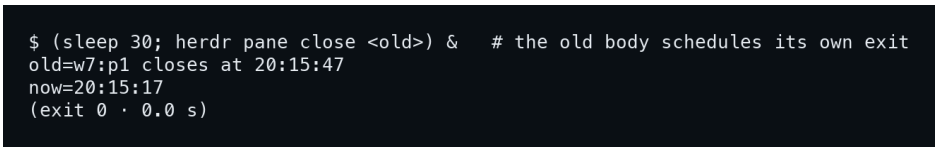
ผลลัพธ์บอก `new=w7:p3` และบรรทัดข้างล่างระบุ cwd เป็น `/opt/Code/github.com/Caeruleus-Lab/aiexpo-oracle` **pane id เปลี่ยนจากตอนอยู่ใน session จริง เพราะ `bookdemo` เป็นคนละ session** ร่างเก่าในรอบนี้คือ w7:p1 ตามที่ภาพถัดไปจะยืนยัน

4.3 ร่างเก่าตั้งเวลาออกเอง

ขั้นที่ 1 — ตั้ง timer 30 วินาทีแล้วนับเวลา

คืนจริงใช้ 40 กับ 20 วินาที ส่วนใน replay ใช้ 30 วินาทีครั้งเดียว

```
(sleep 30; herdr pane close <old>) &   # the old body schedules its own
exit
```



รูปที่ 20: timer 30 วินาที: ตอนนี้ 20:15:17 ร่างเก่าจะปิดตอน 20:15:47

```
old=w7:p1 closes at 20:15:47
now=20:15:17
```

ระหว่าง 20:15:17 ถึง 20:15:47 ร่างเก่ายังมีชีวิตอยู่ ทั้งที่ร่างใหม่เริ่มแล้ว คืนจริงก็เป็นแบบนี้ ร่างเก่ารอ 40 วินาทีแล้วค่อย `/exit` ช่วงรอนั้นมีสอง agent ใน space เดียวกัน และถ้ามีใครสั่ง `herdr-room in` ตอนนั้น มันจะนับเห็นทั้งคู่

ทำไมห้องถึงได้ 3 คอลัมน์แทนที่จะเป็น 2 บทหน้าเปิดดูตรงนั้นพอดี

บทที่ 5: ทำไมมี 3 คอลัมน์

ร่างใหม่เกิดแล้ว (w7:p3) ร่างเก่ายังไม่ปิด (w7:p1) ทีนี้ถ้าสั่ง `herdr-room in` ตอนนี้ จะได้กี่คอลัมน์ คนที่ตั้งใจให้เหลือสองก็คงคิดว่าสอง แต่ห้องนับเองอีกแบบ ทุกภาพในบทนี้มาจากการซ้อมใน session bookdemo (id จึงเป็น w7) คืนจริงตอน 20:05:38 ก็ได้ 3 members เหมือนกัน บทนี้ดูว่าห้องนับยังไง แล้วทำไมแก้ไฟล์ state ก็ไม่ช่วย

5.1 สั่ง in ด้วยสองสมาชิก ได้ห้องสาม

ขั้นที่ 1 — สั่ง room in ด้วยร่างใหม่กับ worktree

พิมพ์ชื่อห้อง แล้วตามด้วยสมาชิกที่ต้องการ คือร่างใหม่ (path ของ main checkout) กับ worktree ของ aiexpo-wt ใส่ `--layout cols` ให้เรียงเป็นคอลัมน์ และ `--session bookdemo` ให้รันใน session ซ้อม ตรงนี้ร่างเก่ายังเปิดอยู่ใน space เดียวกัน

```
herdr-room in aiexpo-oracle <new> <wt> --layout cols --session bookdemo
```

```
$ herdr-room in aiexpo-oracle <new> <wt> --layout cols --session bookdemo
room 'aiexpo-oracle' · session bookdemo · layout cols · 3 members: aiexpo-oracle, aiexpo-wt, …
… aiexpo-oracle
  name  w7:p3 = aiexpo-oracle   (not an agent: skipped)
  name  w7:p2 = aiexpo-wt
  name  w7:p1 = aiexpo-oracle
  saved /Users/beta/.local/state/herdr-room/bookdemo__aiexpo-oracle.json
  rect  w7:p3    x=0    y=0   14x44
  rect  w7:p2    x=14   y=0   14x44
  rect  w7:p1    x=28   y=0   13x44
(exit 0 · 0.1 s)
```

รูปที่ 21: ผลลัพธ์ herdr-room in บอก 3 members และ rect สามก้อนกว้าง 14, 14, 13

สั่งไปสองสมาชิก แต่หัวบรรทัดแรกรายงานว่า **ห้องมี 3 members** และรายชื่อมี aiexpo-oracle สองครั้ง (ร่างใหม่กับร่างเก่า) ไล่ดู rect ก็ได้สามก้อน

ภาพแรกของบทมีบรรทัด `name w7:p3 = aiexpo-oracle (not an agent: skipped)` สาเหตุ capture ไม่ได้บอก บทนี้ไม่เดา แต่ **pane นี้ยังเป็นสมาชิกของห้อง** เพราะบรรทัด `rect w7:p3` ยังอยู่ บทหน้าแก้ที่ต้นเหตุ

```
room 'aiexpo-oracle' · session bookdemo · layout cols · 3 members: …
  rect  w7:p3    x=0    y=0   14x44
```

```
rect  w7:p2    x=14   y=0   14x44
rect  w7:p1    x=28   y=0   13x44
```

กว้าง 14, 14 และ 13 สูง 44 เท่ากันหมด รวมแล้ว 14+14+13 = 41 ก้อนที่สามคือ w7:p1 ร่างเก่า

5.2 ไฟล์ state ก็จดไว้สามคน

ขั้นที่ 1 — เปิดไฟล์ state ดูสมาชิก

ห้องเก็บรายชื่อสมาชิกไว้ในไฟล์ JSON ใช้ `jq` ดึงออกมาทีละคน เอาแค่ pane, name และ path

```
jq -c '.members[]|[.pane,.name,.path]' <state>
```

```
$ jq -c '.members[]|[.pane,.name,.path]' <state>
["w7:p3","aiexpo-oracle","Caeruleus-Lab/aiexpo-oracle"]
["w7:p2","aiexpo-wt","Caeruleus-Lab/aiexpo-oracle/wt/workshop-for-ai-aiexpo-issue3-30oct-fri20 …
… 26"]
["w7:p1","aiexpo-oracle","laris-co/course-team/wt/aiexpo-oracle"]
(exit 0 · 0.0 s)
```

รูปที่ 22: ไฟล์ state มีสมาชิก 3 แถว แถวสุดท้ายเป็น path ของ worktree ร่างเก่า

มีสามแถว สองแถวแรกคือ w7:p3 กับ w7:p2 ที่เราสั่ง ส่วนแถวที่สามคือ **ร่างเก่าที่ห้องเก็บเข้ามาเอง**

```
["w7:p1","aiexpo-oracle","laris-co/course-team/wt/aiexpo-oracle"]
```

path นี้คือ worktree เดิมใน course-team ไม่ใช่ main checkout ตรงนี้ชัดว่าห้องรู้จักร่างเก่า ทั้งที่เราไม่ได้พิมพ์ถึงมันสักคำ

5.3 แก้ state ให้เหลือสอง ไม่ได้อะไรเลย

ขั้นที่ 1 — ตัดร่างเก่าออกจากไฟล์ แล้วสั่ง in อีกรอบ

ความคิดแรกของใครก็คงเป็นแบบนี้ ถ้าห้องอ่านรายชื่อจากไฟล์ ก็ลบแถวร่างเก่าออกจากไฟล์ แล้วสั่งซ้ำ การทดลองนี้ใช้ `jq` ตัดสมาชิกที่ path เป็นร่างเก่า เขียนทับไฟล์ state แล้วรัน `herdr-room in` ชุดเดิม

```
jq 'drop old from .members' > state
herdr-room in aiexpo-oracle <new> <wt> --layout cols --session bookdemo
```

```
$ jq 'drop old from .members' > state; herdr-room in ... again
state now: 2 members
room 'aiexpo-oracle' · session bookdemo · layout cols · 3 members: aiexpo-oracle, aiexpo-wt, …
… aiexpo-oracle
  rect  w7:p3    x=0    y=0   14x44
  rect  w7:p2    x=14   y=0   14x44
  rect  w7:p1    x=28   y=0   13x44
(exit 0 · 0.1 s)
```

รูปที่ 23: ไฟล์ state เหลือ 2 members แต่ room in ยังรายงาน 3 members และ rect เดิม

capture บอกว่า `state now: 2 members` แต่บรรทัดถัดมายังเป็น `3 members` และ rect ยังเป็น 14, 14, 13 เหมือนเดิมทุกตัว **การแก้ไฟล์ state ไม่ได้ลบ pane ออกจากห้อง** ส่วนทำไมถึงเป็นแบบนี้ ต้องไปอ่านโค้ด

5.4 โค้ดที่ตัดสินว่าใครอยู่ในห้อง

ขั้นที่ 1 — อ่านบรรทัด 235 ถึง 250 ของ herdr-room

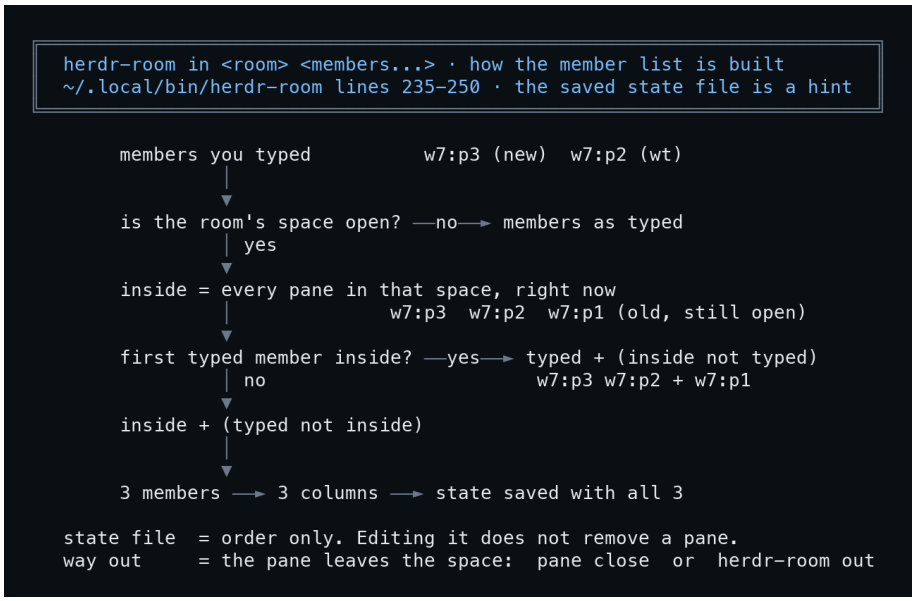
เปิดสคริปต์ด้วย `sed -n` ดูเฉพาะช่วงที่เกี่ยวกับห้องที่เปิดอยู่

```
sed -n 235,250p ~/.local/bin/herdr-room
```

```
$ sed -n 235,250p ~/.local/bin/herdr-room
    if ws:   # the room is open: keep everyone inside it; new members join at the end
        order_paths = [m["path"] for m in (state or {}).get("members", [])]
        pos = {}
        for l in snap.get("layouts", []):
            for lp in l.get("panes", []):
                pos[lp["pane_id"]] = (lp["rect"]["x"], lp["rect"]["y"])
        inside = [p for p in snap.get("panes", []) if p["workspace_id"] == ws["workspace_id"]]
        inside.sort(key=lambda p: (order_paths.index(real(p.get("cwd"))) if …
        … real(p.get("cwd")) in order_paths else 99,
                                   pos.get(p["pane_id"], (0, 0))))
        existing = [{"pane": p["pane_id"], "path": real(p.get("cwd")),
                     "name": names.get(real(p.get("cwd"))) or name_of(p, agents), "ws": …
                     … p["workspace_id"]} for p in inside]
        ids = {m["pane"] for m in existing}
        if members and members[0]["pane"] in ids:      # the caller's order wins when it …
        … starts inside the room
            members = members + [m for m in existing if m["pane"] not in {x["pane"] for x in …
            … members}]
        else:
            members = existing + [m for m in members if m["pane"] not in ids]
(exit 0 · 0.0 s)
```

รูปที่ 24: ซอร์ส herdr-room บรรทัด 235 ถึง 250

การ์ดนี้วาดเส้นทางตัดสินใจของโค้ดสิบหกบรรทัดนี้ไว้ก่อน



รูปที่ 25: แผนภาพ D3 กฎการสร้างรายชื่อสมาชิกของ herdr-room in

แล้วอ่านทีละบรรทัดได้ดังนี้

- บรรทัด 235 `if ws:` ถ้า space ของห้องเปิดอยู่ (มี workspace ตัวจริง) ก็เข้าสาขานี้ คอมเมนต์บอกว่า “keep everyone inside it”
- บรรทัด 236 `order_paths` ดึง path จาก `members` ใน state เก็บไว้เป็น **ลำดับ** อย่างเดียว
- บรรทัด 241 `inside` คือ pane ทุกตัวที่ `workspace_id` ตรงกับ space ของห้อง ณ ตอนนี้ จุดนี้แหละที่ร่างเก่าเข้ามา เพราะมันยังนั่งอยู่ใน space
- บรรทัด 237–243 จดตำแหน่งบนจอ แล้วเรียง `inside` ตามลำดับใน state
- บรรทัด 244–245 ทำ `existing` ซึ่งเป็นรายการสมาชิกที่สร้างจาก pane จริง มี pane, path, ชื่อ และ workspace
- บรรทัด 246 `ids` เก็บ id ของ pane ที่อยู่ข้างในทั้งหมด
- บรรทัด 247–250 ถ้าสมาชิกคนแรกที่พิมพ์อยู่ข้างในห้องอยู่แล้ว ลำดับที่พิมพ์ชนะ แล้วต่อท้ายด้วย pane ข้างในที่ไม่ได้พิมพ์ ไม่งั้นเอา `existing` ขึ้นก่อน แล้วค่อยเติมคนที่พิมพ์แต่ยังไม่อยู่ข้างใน

ในเคสเรา w7:p3 ที่พิมพ์เป็นคนแรกอยู่ข้างในห้อง จึงเป็น `typed + (inside not typed)` และ w7:p1 ก็ตามมาต่อท้าย **ไฟล์ state เป็นแค่คำใบ้เรื่องลำดับ ตัวตัดสินสมาชิกคือ pane ที่อยู่ใน space จริง** ทางออกจึงอยู่ในกล่องสุดท้ายของการ์ด คือให้ pane ออกจาก space ด้วย `pane close` หรือ `herdr-room out` ไม่ใช่แก้ไฟล์

บทที่ 6: ทางแก้ — ให้ร่างเก่าออกก่อน

บทที่ 5 เห็นแล้วว่าห้องมี 3 คอลัมน์เพราะ `herdr-room in` ไล่เก็บ pane ทุกอันที่อยู่ในห้อง ตอนนั้นร่างเก่ายังไม่ไปไหน ทางแก้เลยไม่ใช่การแก้ไฟล์ state แต่คือให้ร่างเก่าออกจากห้องก่อน แล้วค่อยเรียก room in ใหม่ บทนี้ทำให้ดูใน session `bookdemo` ทีละขั้น

6.1 ทางออกมีสองทาง

ร่างเก่าออกจากห้องได้สองทาง ทางแรกคือสั่ง `herdr-room out` ทางที่สองคือปิด pane ทิ้งเลย ลองทางแรกด้วย `--dry` ก่อน จะได้เห็นว่ามันตั้งใจทำอะไรบ้างโดยที่ยังไม่แตะอะไรจริง

ขั้นที่ 1 — ดู dry run ของ out

สั่ง out กับชื่อ `aiexpo-oracle` และ path ของร่างเก่า (ในภาพเขียนว่า `<old>`) พร้อม `--dry` คำสั่งนี้ไม่ย้ายอะไร แค่พิมพ์แผนออกมา

```
herdr-room out aiexpo-oracle <old> --session bookdemo --dry
```

```
$ herdr-room out aiexpo-oracle <old> --session bookdemo --dry   (the other way)
room 'aiexpo-oracle' · session bookdemo · 1 member(s) out: aiexpo-oracle
  plan  herdr --session bookdemo pane move w7:p1 --new-workspace --label aiexpo-oracle …
  … --no-focus   # aiexpo-oracle to a space of its own
  plan  herdr --session bookdemo worktree open --cwd …
  … /opt/Code/github.com/laris-co/course-team --path …
  … /opt/Code/github.com/laris-co/course-team/wt/aiexpo-oracle --label aiexpo-oracle …
  … --no-focus   # nest aiexpo-oracle under its repo
  plan  herdr --session bookdemo workspace move <aiexpo-oracle's space> --after <last space …
  … of course-team or w7>   # aiexpo-oracle beside its repo
  --dry: nothing was moved
(exit 0 · 0.1 s)
```

รูปที่ 26: dry run ของ herdr-room out: แผน 3 บรรทัด

แผนมี 3 บรรทัด บรรทัดแรกย้าย `w7:p1` ไปเป็น space ของตัวเอง ซึ่งก็คือสิ่งที่ต้องการ แต่บรรทัดที่สองคือ

`worktree open --cwd .../laris-co/course-team --path .../course-team/wt/aiexpo-oracle` แปลว่า out จะ **เปิด space ของ worktree ใต้ course-team** เพิ่มให้ด้วย ซึ่งไม่ใช่สิ่งที่ต้องการ เพราะเป้าหมายคือให้ร่างเก่าหายไป ไม่ใช่ให้มีที่อยู่ใหม่ใต้ repo เดิม

ขั้นที่ 2 — เลือกปิด pane

คืนจริงไม่ได้ใช้ out แต่ใช้ `pane close` ตามที่ร่างเก่าตั้ง timer ไว้เอง (บทที่ 4) พอ timer ครบ ร่างเก่าก็ `/exit` แล้วปิด pane ของตัวเอง ในรีเพลย์ timer ตั้งไว้ 30 วินาที เหตุผลที่ **ปิด pane ได้ตรงกว่า out** คือไม่มีผลข้างเคียง ไม่มี space ใหม่โผล่มา ข้อเสียมีข้อเดียว คือต้องรอให้ pane หายไปจริงก่อนทำขั้นต่อไป

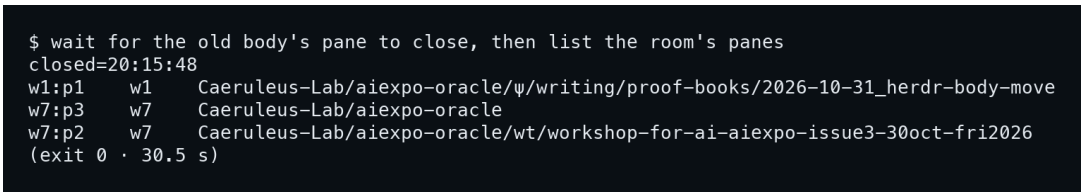
6.2 รอจนร่างเก่าหาย

ห้ามรีบเรียก room in ทันทีหลังสั่งปิด เพราะบทที่ 5 พิสูจน์แล้วว่าแค่ pane ยังอยู่ ห้องก็ยังนับมัน จึงต้องรอแล้วตรวจรายการ pane ก่อน

ขั้นที่ 1 — รอแล้วดูรายการ pane

รอจน pane ของร่างเก่าปิด จดเวลาที่เห็น แล้ว list pane ของห้องใน session `bookdemo` (ภาพนี้คือผลของการรอแล้ว list ไม่ใช่คำสั่งเดียว) pane ที่ควรเหลือมีสองอัน

```
w7:p3   w7  Caeruleus-Lab/aiexpo-oracle
w7:p2   w7  Caeruleus-Lab/aiexpo-oracle/wt/workshop-for-ai-aiexpo-
issue3-…
```



รูปที่ 27: เวลาที่ pane ปิด และรายการ pane ที่เหลือ

ภาพบอกว่าปิดตอน `closed=20:15:48` และคำสั่งรอใช้เวลา 30.5 วินาที ตรงกับ timer 30 วินาที รายการที่เหลือในห้องมีแค่ 2 pane ตามที่ยกมาข้างบน

ส่วน `w1:p1` ที่โผล่บรรทัดบนสุดเป็น pane ที่ใช้เขียนหนังสือเล่มนี้ ใน space `w1` อีกอัน ไม่ได้อยู่ในห้อง ร่างเก่าไม่อยู่ในรายการแล้ว ใช้ **รายการ pane เป็นตัวตัดสิน** ไม่ใช่ความรู้สึกว่า timer น่าจะครบ

6.3 room in อีกรอบ

พอร่างเก่าไม่อยู่ ห้องก็เหลือแค่ร่างใหม่กับ worktree partner เรียก room in ซ้ำได้เลย

ขั้นที่ 1 — สั่ง room in ด้วยสองสมาชิก

ส่งแค่สองตัว คือร่างใหม่ (`<new>`) กับ worktree (`<wt>`) ใช้ layout `cols` เหมือนรอบแรก

```
herdr-room in aiexpo-oracle <new> <wt> --layout cols --session bookdemo
```

```
$ herdr-room in aiexpo-oracle <new> <wt> --layout cols --session bookdemo
room 'aiexpo-oracle' · session bookdemo · layout cols · 2 members: aiexpo-oracle, aiexpo-wt
  name  w7:p3 = aiexpo-oracle
  name  w7:p2 = aiexpo-wt
  saved /Users/beta/.local/state/herdr-room/bookdemo__aiexpo-oracle.json
  rect  w7:p3    x=0    y=0   21x44
  rect  w7:p2    x=21   y=0   20x44
(exit 0 · 0.1 s)
```

รูปที่ 28: room in รอบที่สอง: 2 สมาชิก กว้าง 21 กับ 20

หัวผลลัพธ์บอก `2 members: aiexpo-oracle, aiexpo-wt` รอบนี้บรรทัด `name` ขึ้นครบทั้งสองตัว ไม่มีคำว่า skipped ซึ่งต่างจากบทที่ 5 ที่ร่างใหม่ถูกข้ามตอนตั้งชื่อ ขนาดคอลัมน์เป็น

```
rect  w7:p3    x=0    y=0   21x44
rect  w7:p2    x=21   y=0   20x44
```

สองคอลัมน์ กว้าง 21 กับ 20 สูง 44 เท่ากัน แทนที่จะเป็น 3 คอลัมน์แคบ ๆ แบบเดิม

6.4 state กับ list ต้องตรงกัน

แค่หน้าจอถูกยังไม่พอ ต้องเช็กว่าไฟล์ state ที่ room in บันทึกไว้ บอกเรื่องเดียวกับที่ห้องเป็นจริง เพราะรอบหน้า `in` จะอ่านจากมัน

ขั้นที่ 1 — อ่าน state แล้ว list ห้อง

ใช้ `jq` ดึงสมาชิกจากไฟล์ state แล้วเรียก `herdr-room list` ต่อท้าย

ผลที่จะเห็น: state ไม่มี path ของร่างเก่าแล้ว saved ลงท้าย `20:15:48` ตรงกับตอน pane เก่าปิด **state กับ list ตรงกัน** บทหน้าเปิด transcript ของคืนจริง

```
jq -c '.members[]|[.pane,.name,.path]' <state>; herdr-room list
```



รูปที่ 29: state มี 2 สมาชิก และ list เห็นห้อง OPEN

บทที่ 7: คืนจริง และซ้ายคุยขวา

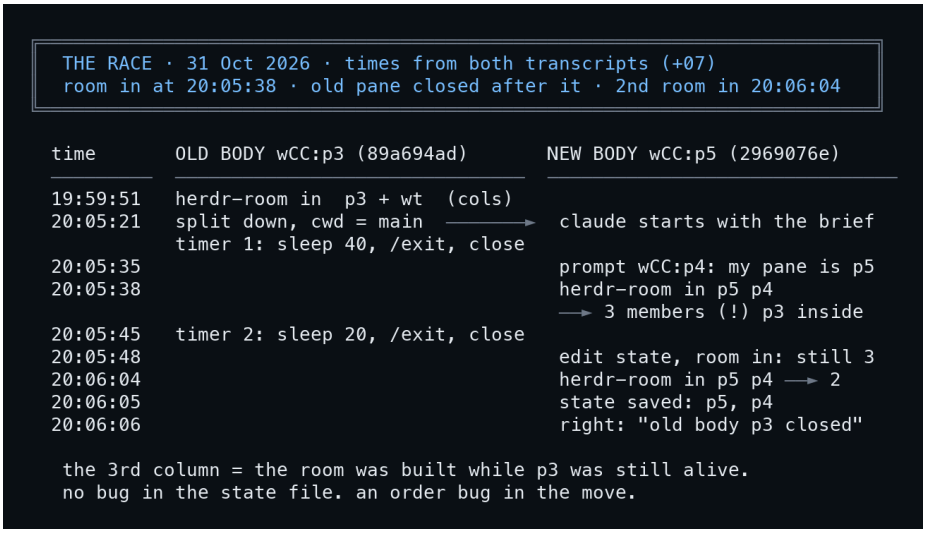
บทก่อนหน้าเป็นการเล่นซ้ำใน session `bookdemo` ทั้งหมด บทนี้กลับไปที่คืนจริงของ 31 ต.ค. ที่ห้อง `wCC` ใน session `laris-co` เราไม่ได้รันอะไรเพิ่มในห้องจริง แค่อ่านย้อนจาก transcript ของ agent สองตัว แล้วดูว่ากฎของทีมที่ตามมาหลังย้ายเสร็จคืออะไร

7.1 การแข่งกันของสองร่าง

ปัญหา 3 คอลัมน์ในบทที่ 5 ไม่ใช่บั๊กในไฟล์ state แต่เป็นเรื่องลำดับเวลา ภาพแรกของบทนี้วางเหตุการณ์ของทั้งสองร่างลงบนแกนเวลาเดียวกัน ให้เห็นว่าใครทำอะไรก่อนหลัง

ขั้นที่ 1 — ดูแกนเวลาจาก transcript ทั้งสองฝั่ง

ภาพนี้เป็นการ์ดที่เราวาดเองจากเวลาใน transcript (เวลาไทย UTC+7) ไม่ใช่ผลของคำสั่งตัวเดียว คอลัมน์ซ้ายคือร่างเก่า `wCC:p3` (89a694ad) คอลัมน์ขวาคือร่างใหม่ `wCC:p5` (2969076e)



รูปที่ 30: การ์ดแกนเวลา ร่างเก่ากับร่างใหม่ ตั้งแต่ 19:59:51 ถึง 20:06:06

อ่านจากบนลงล่าง ตอน 20:05:21 ร่างเก่า split pane ลงมาแล้วสตาร์ท claude ในโฟลเดอร์ main พร้อมตั้งเวลาปิดตัวเองไว้ 40 วินาที ตอน 20:05:38 ร่างใหม่รัน `herdr-room in` เป็นครั้งแรก ตอนนั้น p3 ยังมีชีวิตอยู่ในห้อง เลยได้สมาชิก 3 คน

> **ห้องถูกสร้างตอนที่ร่างเก่ายังไม่ปิด** ลำดับการย้ายผิด ไม่ใช่ไฟล์ state ผิด

พอ 20:06:04 p3 ปิดไปแล้ว `herdr-room in` จึงได้ 2 สมาชิก และ 20:06:05 บันทึก state เป็น p5 กับ p4 ตรงกับสิ่งที่บทที่ 6 แก้ไว้ในการเล่นซ้ำ คือให้ร่างเก่าออกก่อนแล้วค่อยสร้างห้อง

7.2 สอง transcript เคียงข้างกัน

เครื่องมืออ่านย้อนคือ `timeline.py` (บทที่ 8 จะเล่าว่าทำงานยังไง) มันพิมพ์หนึ่งบรรทัดต่อหนึ่งเหตุการณ์ ขึ้นต้นด้วยเวลา แล้วบอกว่าเป็น `run` (คำสั่งที่ agent รัน) หรือ `msg` (ข้อความที่เข้ามา) ข้อควรรู้คือเครื่องมือตัดบรรทัดยาวด้วย “…” ดังนั้นเนื้อคำสั่งเต็มจะไม่อยู่ในภาพ

ขั้นที่ 1 — อ่านบันทึกของร่างเก่า

ให้กรองช่วง 19:54 ถึง 20:06 ด้วย `--since` กับ `--until`

```
python3 timeline.py <89a694ad, old body> --since 19:54 --until 20:06
```

```
$ python3 timeline.py <89a694ad, old body> --since 19:54 --until 20:06
19:54:08  msg   [right wCB:p2] PR #5 ready for your merge: Caeruleus-Lab/aiexpo-oracle#5 …
… (branch wor …
19:56:24  run   herdr agent prompt wCC:p2 "[wCB:p1 → wCC:p2] Correction from Nat: your place …
… is the  …
19:57:02  run   herdr agent prompt wCC:p2 "[wCC:p3 → wCC:p2] Nat chose this: I moved my own …
… pane int …
19:59:43  run   herdr-room in aiexpo-room wCC:p3 wCE:p2 --layout cols --session laris-co …
… --dry 2>&1  …
19:59:49  run   herdr-room in aiexpo-oracle wCC:p3 wCE:p2 --layout cols --session laris-co …
… --dry 2>& …
19:59:51  run   herdr-room in aiexpo-oracle wCC:p3 wCE:p2 --layout cols --session laris-co …
… 2>&1 | ta …
19:59:55  run   herdr agent prompt wCC:p4 "[wCC:p3 → wCC:p4] Nat said 'room in': you are now …
… wCC:p4, …
20:05:21  run   BRIEF="You are AIExpo Oracle, body back in your own main checkout ($R), herdr …
… pane $ …
20:05:21  run   herdr pane close wCC:p3) >/dev/null 2>&1 &! claude \"$BRIEF\"" 2>&1 | python3 …
… -c 'im …
20:05:35  msg   [right wCC:p4 → left wCC:p3] Nat: 'keep talk left right'. Sync so we don't …
… overlap.  …
20:05:41  run   herdr agent prompt wCC:p4 "[wCC:p3 → wCC:p4] I'm the old body and closing
20:05:45  run   herdr pane close wCC:p3" >/dev/null 2>&1 &
(exit 0 · 0.0 s)
```

รูปที่ 31: บันทึกเวลาของร่างเก่า 89a694ad ตั้งแต่ 19:54 ถึง 20:06

มีสามบรรทัดที่ต้องจับ ตอน 19:59:51 มันรัน `herdr-room in aiexpo-oracle wCC:p3 wCE:p2` ครั้งแรกเพื่อรวมตัวเองกับ worktree เข้าห้อง สังเกตว่า worktree ตอนนั้นคือ `wCE:p2` ไม่ใช่ `p4` เพราะ **pane id เปลี่ยนตอนย้าย** ถัดมา 20:05:21 คือคำสั่งสร้างร่างใหม่ และ 20:05:45 คือคำสั่งที่ตั้งตัวจับเวลาตัวที่สองไว้ปิด `wCC:p3` (sleep 20 แล้วค่อย close)

ขั้นที่ 2 — อ่านบันทึกของร่างใหม่

ร่างใหม่รันคำสั่งเดียวกัน แต่ใช้ `--until 20:07` เพราะเพิ่งเกิดตอน 20:05

```
python3 timeline.py <2969076e, new body> --until 20:07
```

```
$ python3 timeline.py <2969076e, new body> --until 20:07
20:05:35  run   herdr --session laris-co agent prompt wCC:p4 '[aiexpo-oracle] Body moved back …
… to mai …
20:05:37  msg   [wCC:p3 → wCC:p5] Nat: 'keep talk left right'. After the room re-layout, …
… wCC:p5 (new …
20:05:38  run   herdr-room in aiexpo-oracle wCC:p5 wCC:p4 --layout cols --session laris-co …
… 2>&1 | ta …
20:05:48  run   jq '.members |= map(select(.pane!="wCC:p3"))' "$f.bak-3members" > "$f"
20:05:48  run   herdr-room in aiexpo-oracle wCC:p5 wCC:p4 --layout cols --session laris-co …
… 2>&1 | rg …
20:06:04  run   herdr-room in aiexpo-oracle wCC:p5 wCC:p4 --layout cols --session laris-co …
… 2>&1 | rg …
20:06:06  msg   [right wCC:p4 → left wCC:p5] Old body wCC:p3 (89a694ad) closed — it was idle …
… and sai …
20:06:14  run   herdr --session laris-co agent prompt wCC:p4 '[left wCC:p5 → right wCC:p4] …
… Ack: sole …
(exit 0 · 0.0 s)
```

รูปที่ 32: บันทึกเวลาของร่างใหม่ 2969076e ตั้งแต่เกิดจนถึง 20:06:14

บรรทัดที่ 20:05:37 เป็นข้อความที่ Nat ฝากมาผ่านร่างเก่า คือ `keep talk left right` จากนั้น 20:05:38 คือ `herdr-room in` รอบที่ได้ 3 สมาชิก ตามด้วย `jq` ตัด p3 ออกจาก state ตอน 20:05:48 แล้วรัน room in ซ้ำก็ยังได้ 3 เหมือนเดิม (ผลรอบนั้นไม่ได้เก็บไว้ในคืนจริง บทที่ 5 ในการเล่นซ้ำให้ผลเดียวกัน) พอ 20:06:04 รันอีกครั้งจึงเหลือ 2 ส่วน บรรทัด 20:06:06 เป็นข้อความจากฝั่งขวา `wCC:p4` ว่าร่างเก่า p3 ปิดแล้ว และ 20:06:14 ร่างใหม่ตอบรับกลับ

7.3 กฎซ้ายคุยขวา

ประโยคของ Nat สั้นมาก คือ keep talk left right ไฟล์ `CLAUDE.md` ของ `laris-team` มีกฎเรื่องนี้อยู่ที่บรรทัด 51

ขั้นที่ 1 — หากฎในไฟล์ด้วย rg

ค้นสองวลีที่เป็นหัวใจของกฎ ใช้ `rg` ในไฟล์เดียว ไม่กวาดทั้ง tree

```
rg -n 'left pane|Pane ids change' laris-team/CLAUDE.md
```

```
$ rg -n 'left pane|Pane ids change' laris-team/CLAUDE.md
50:- Pane ids change on every move: find agents by cwd. One-line `herdr agent prompt`s only.
51:- In a room the left pane talks with Nat; the right panes report to it.
(exit 0 · 0.0 s)
```

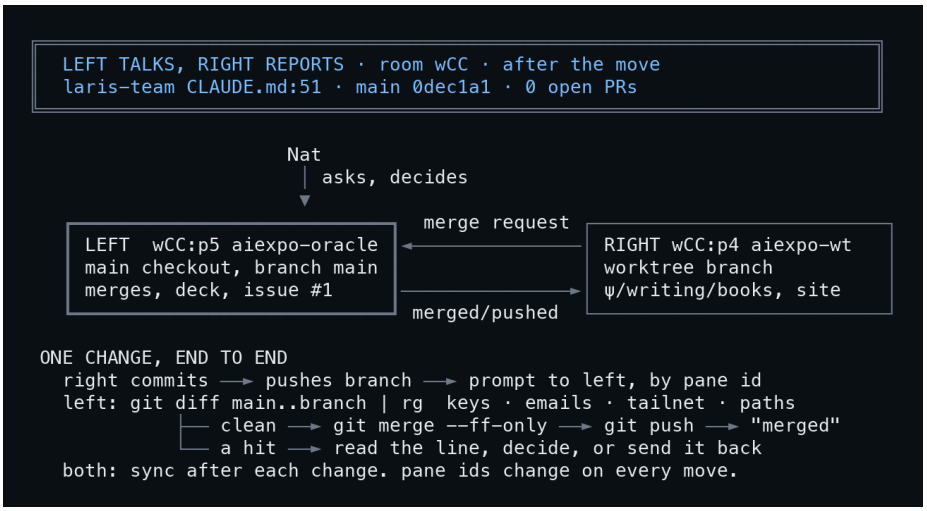
รูปที่ 33: ผล rg พบกฎสองบรรทัดที่ 50 และ 51 ใน CLAUDE.md

```
50:- Pane ids change on every move: find agents by cwd. One-line …
51:- In a room the left pane talks with Nat; the right panes report to
it.
```

สองบรรทัดนี้สอนสองเรื่อง บรรทัด 51 บอกว่า **ซ้ายคุยกับ Nat ขวารายงานซ้าย** ตามการ์ดและลำดับ argument ของ `herdr-room in` ห้องของเราจึงมี p5 (main) อยู่ซ้าย และ p4 (worktree) อยู่ขวา ส่วนบรรทัด 50 ให้หา agent จาก cwd อย่าจำเลข `p3` ไว้

ขั้นที่ 2 — ดูการ์ดว่าใครทำอะไร

การ์ดนี้สรุปบทบาทของสอง pane กับเส้นทางของงานหนึ่งชิ้น ตั้งแต่ commit จนถึง merge ซึ่งข้อความเดียวอธิบายได้ไม่ชัดเท่ารูป



รูปที่ 34: การ์ดซ้ายคุยขวารายงาน พร้อมลำดับงานหนึ่งชิ้นจาก commit ถึง merged

ฝั่งขวา **commit ใน worktree แล้ว push branch** จากนั้น prompt หาฝั่งซ้ายด้วย pane id ฝั่งซ้ายตรวจ diff ถ้าสะอาดก็ merge แบบ fast-forward ถ้าเจอบรรทัดน่าสงสัยก็อ่านก่อนตัดสินใจ หรือส่งกลับไปแก้ ทั้งสองฝั่งต้อง sync กันหลังทุกการเปลี่ยนแปลง

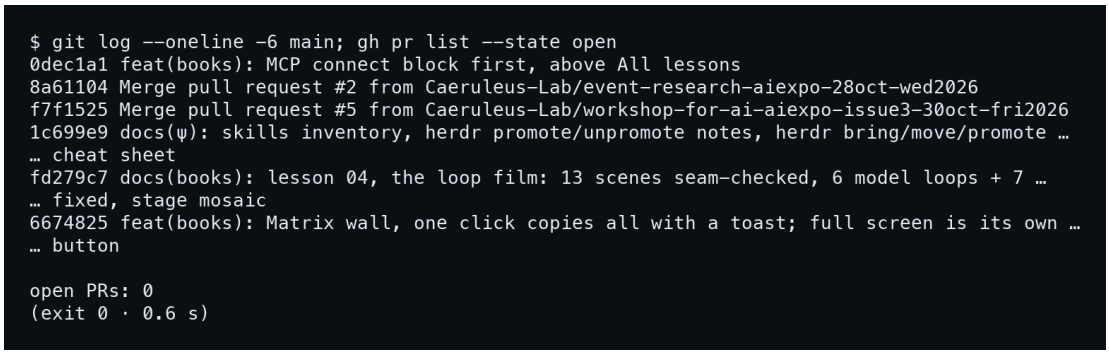
7.4 หนึ่ง merge ตั้งแต่ต้นจนจบ

ในคืนนั้นฝั่งซ้าย merge ไปสามรอบ คือ `0dec1a1`, `8881cc1` และ `53c5e38` ทุกรอบเป็น fast-forward หลังสแกน ภาพที่เราเก็บไว้ตรงกับรอบแรก `0dec1a1` ส่วนสองรอบหลังเกิดตามมาหลังเก็บภาพ

ขั้นที่ 1 — ดู log ของ main และ PR ที่ค้าง

ก่อนและหลัง merge ให้เช็กสองอย่าง คือประวัติบน main กับจำนวน PR ที่ยังเปิด

```
git log --oneline -6 main; gh pr list --state open
```



รูปที่ 35: git log 6 บรรทัดล่าสุดของ main และผล open PRs เป็น 0

```
0dec1a1 feat(books): MCP connect block first, above All lessons
8a61104 Merge pull request #2 from Caeruleus-Lab/event-research-aiexpo-…
…
open PRs: 0
```

บนสุดคือ `0dec1a1` ที่เพิ่ง merge และท้ายสุดบอก `open PRs: 0` แปลว่า**ไม่มี PR ที่เปิดค้าง**

ขั้นที่ 2 — สแกน diff ด้วยสี่แบบ

สแกนนี้รันบน diff ของช่วงก่อนหน้า `fd279c7..8a61104` ด้วย `rg -c` นับบรรทัดที่ตรงกับ pattern สี่ชุด คือ key, อีเมล, tailnet และ path ในเครื่อง

```
git diff fd279c7..8a61104 | rg -c <keys|emails|tailnet|local paths>
```



รูปที่ 36: ผลสแกนสี่ pattern: keys 0, emails 0, tailnet 0, local paths 3

```
sk-[A-Za-z0-9]{16,}|ghp_[A-Za-z0-9]{20,}|AKIA[0-9A-Z]{16}      0
/Users/beta|/opt/Code                                          3
```

สามชุดแรกได้ 0 ทั้งหมด คือ key, อีเมล `@gmail.com` และที่อยู่ tailnet ไม่มีเลย ส่วนชุดสุดท้ายได้ 3 ซึ่ง **ตัวเลขที่ไม่ใช่ 0 ต้องอ่านเป็นบรรทัด ไม่ใช่ตัดสินจากตัวเลข** เรื่องนี้ต่อในหัวข้อถัดไป

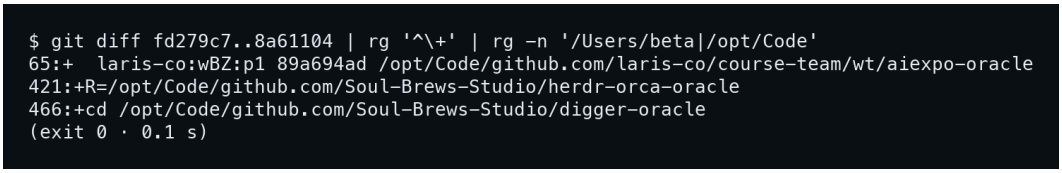
7.5 สามบรรทัดที่สแกนเจอ

ช่วง `fd279c7..8a61104` เป็น range ก่อนหน้า เราจึงกลับไปดูว่าสามบรรทัดนั้นคืออะไร

ขั้นที่ 1 — ดึงบรรทัดที่ตรงออกมาอ่านเอง

คราวนี้ไม่ใช้ `-c` แต่กรองเอาเฉพาะบรรทัดที่เพิ่มเข้ามา (ขึ้นต้นด้วย `+`) แล้วให้ `rg -n` ใส่เลขบรรทัด

```
git diff fd279c7..8a61104 | rg '^\+' | rg -n '/Users/beta|/opt/Code'
```



รูปที่ 37: สามบรรทัดที่ตรง pattern path ในเครื่อง บรรทัด 65, 421, 466

บรรทัด 65 คือรายการ agent ที่มี session id กับโฟลเดอร์ทำงาน บรรทัด 421 กับ 466 คือคำสั่ง `R=` และ `cd` ที่อยู่ในโน้ต vault ทั้งสามเป็น **path ของโฟลเดอร์ในเครื่อง ไม่ใช่ความลับ** ไม่มี token ไม่มี key และเราไม่เรียกมันว่าข้อมูลรั่ว

สแกนมีไว้ให้คนอ่านก่อน merge ไม่ได้ตัดสินแทน บทสุดท้ายกลับไปดูว่าอะไรยังค้าง

บทที่ 8: สิ่งที่ยังค้าง และสูตรย้ายร่าง

ย้ายเสร็จแล้ว ห้องก็เหลือสองคอลัมน์ แต่งานจริงไม่เคยจบเรียบร้อยร้อยเปอร์เซ็นต์. บทนี้ไล่ดูว่าอะไรถูกต้องแล้ว อะไรยังค้างอยู่บนดิสก์ และอะไรเป็นของคนอื่นที่เราไม่ควรแตะ. ท้ายบทเก็บทั้งเล่มเป็นสูตรการ์ดใบเดียว.

8.1 สถานะห้องจริงถูกต้องแล้ว

ขั้นที่ 1 — อ่านไฟล์ state ของห้องจริง

ห้องจริงชื่อ `aiexpo-oracle` อยู่ใน session `laris-co`. อ่านไฟล์ state ด้วย `jq` เหมือนที่ทำกับห้องจำลองในบทที่ 3 แล้วดูสองอย่าง คือ members กับเวลา `.saved` (`<laris-co state>` คือ path ของไฟล์ state ห้องนี้ ย่อไว้ตามที่ capture แสดง). ครั้งนี้เป็นการอ่านอย่างเดียว ไม่ได้ย้ายอะไรในห้องจริง.

```
jq -c '.members[]|[.pane,.name,.path]' <laris-co state>
jq -r .saved <laris-co state>
```

```
$ jq -c '.members[]|[.pane,.name,.path]' <laris-co state>; jq -r .saved <laris-co state>
["wCC:p5","aiexpo-oracle","Caeruleus-Lab/aiexpo-oracle"]
["wCC:p4","aiexpo-wt","Caeruleus-Lab/aiexpo-oracle/wt/workshop-for-ai-aiexpo-issue3-30oct-fri2 …
… 026"]
2026-10-31T20:06:05+0700
(exit 0 · 0.0 s)
```

รูปที่ 38: state ของห้องจริง มีสอง members คือ wCC:p5 กับ wCC:p4 และเวลา saved

มีสอง members พอดี. `wCC:p5` คือ `aiexpo-oracle` ที่ชี้ไป `Caeruleus-Lab/aiexpo-oracle` (ร่างใหม่ใน main) กับ `wCC:p4` คือ `aiexpo-wt` ที่ชี้ไป worktree ของงาน issue3. **ไม่มี p3 แล้ว** และเวลา saved คือ `2026-10-31T20:06:05+0700` ตรงกับรอบ `room in` ที่รันตอน 20:06:04 หลังร่างเก่าปิดไปแล้ว (รอบก่อนหน้าตอน 20:05:48 ที่ลบ p3 ออกจากไฟล์แล้วยังได้ 3).

8.2 ไดเรกทอรีร่างเก่ายังอยู่

ขั้นที่ 1 — ดูว่าร่างเก่าทิ้งอะไรไว้บนดิสก์

ปิด pane ไม่ได้แปลว่าโฟลเดอร์หายไป. pane คือร่าง ส่วนโฟลเดอร์ `course-team/wt/aiexpo-oracle` เป็นแค่ checkout ที่ร่างเคยยืนอยู่. ลอง `ls` แล้วนับไฟล์ที่ยังไม่ commit ด้วย `git status --short | wc -l`. `<old body dir>` กับ `<old>` คือโฟลเดอร์ร่างเก่า ย่อไว้ตามที่ capture แสดง ส่วนบรรทัดสุดท้ายคือการนับ agent ใน `laris-co` ที่ cwd ตรงกัน.

```
ls <old body dir>
git -C <old> status --short | wc -l
agents with cwd = <old>
```



รูปที่ 39: เนื้อหาโฟลเดอร์ร่างเก่า และข้อความ uncommitted 0 files

โฟลเดอร์ยังมี `ψ`, `CLAUDE.local.md`, `CLAUDE.md`, `justfile`, `README.md` ครบ ส่วนสามบรรทัดท้ายของภาพ (ป้าย `uncommitted`, `unpushed`, `agents`) capture เติมให้เอง ไม่ใช่ผลของ `wc -l` ตรง ๆ: `uncommitted: 0 files`, `unpushed: 0 commits`, `agents in laris-co with this cwd: 0`. **ไม่มีงานค้างให้เสียดาย** เพราะไม่มีไฟล์ค้าง ไม่มี commit ที่ยังไม่ push และไม่มี agent ใน `laris-co` ผูกกับโฟลเดอร์นี้. จะลบ worktree นี้เมื่อไหร่ก็เป็นเรื่องที่ Nat ตัดสิน ตามกฎ Nothing is Deleted เราแค่รายงานว่ามันสะอาด.

8.3 charter ยังชี้ไปที่เก่า

ขั้นที่ 1 — หาเจ้าของ session ที่ต้องแก้

ในบทที่ 1 เราเห็นแล้วว่า charter ใน `laris-team` ยังเขียนว่าร่างอยู่ที่ `course-team/wt`. ย้ายร่างไม่ได้แก้ไฟล์นั้นให้เอง. คำถามคือใครแก้. ถาม herdr ว่า session `laris-co` มี agent ตัวไหนดูแล `laris-team` อยู่.

```
herdr --session laris-co agent list
```

```
$ herdr --session laris-co agent list | jq (LarisCo host w60:p1)
w60:p1    working    laris-co/laris-team
(exit 0 · 0.0 s)
```

รูปที่ 40: agent ของ LarisCo host ที่ w60:p1 กำลังทำงานใน laris-co/laris-team

ผลคือ `w60:p1` สถานะ `working` ที่ `laris-co/laris-team`. agent ที่ทำงานใน `laris-team` ตัวนี้น่าจะเป็น LarisCo host เจ้าของ charter (ตัวกรอง `jq` ของคำสั่งไม่ได้แสดงใน capture). **การแก้ charter เป็นงานของ host ไม่ใช่ของเรา** เพราะไฟล์อยู่ใน repo ของเขา. สิ่งที่เราทำได้คือส่งข้อความบอก pane `w60:p1` ว่าที่อยู่ใหม่คือ main checkout แล้วปล่อยให้เขาแก้เอง.

8.4 timeline.py อ่านคืนนั้นกลับมาอย่างไร

ขั้นที่ 1 — เปิดสคริปต์ที่ใช้อ่าน transcript

คืนจริงในบทที่ 7 ไม่ได้มาจากความจำ. มันมาจาก transcript ของ Claude Code สองไฟล์ ผ่าน `timeline.py` ที่ยาวไม่ถึงร้อยบรรทัด. เปิดดูทั้งไฟล์ก่อนแล้วค่อยไล่ทีละส่วน.

```
cat timeline.py
```

```
$ cat timeline.py
#!/usr/bin/env python3
"""Print the herdr moves and pane-to-pane messages of one Claude Code session, in order.

Usage: python3 timeline.py <session.jsonl> [--since HH:MM] [--until HH:MM]
Times are converted from the transcript's UTC to Asia/Bangkok (+07).
Only lines that matter to the body move are kept: herdr-room calls, agent prompts,
pane closes, and messages that arrived from another pane ("[wCC:" / "[right" / "[left").
"""
import json, sys, re
from datetime import datetime, timedelta, timezone

BKK = timezone(timedelta(hours=7))
KEEP_CMD = re.compile(r"herdr-room (in|out)|agent prompt wCC|pane close|jq '\.members")
KEEP_MSG = re.compile(r"^\[(wCC:p\d|right|left)")

path = sys.argv[1]
since = sys.argv[sys.argv.index("--since") + 1] if "--since" in sys.argv else "00:00"
until = sys.argv[sys.argv.index("--until") + 1] if "--until" in sys.argv else "99:99"

def when(ts):
    return datetime.fromisoformat(ts.replace("Z", …
    … "+00:00")).astimezone(BKK).strftime("%H:%M:%S")

def short(s, n=86):
    s = " ".join(s.split())
    return s if len(s) <= n else s[: n - 2] + " …"

seen = set()
for line in open(path, encoding="utf-8"):
    try:
        d = json.loads(line)
    except ValueError:
        continue
    ts = d.get("timestamp")
    if not ts or not (since <= when(ts)[:5] < until):
        continue
    msg = d.get("message") or {}
    content = msg.get("content")
    if d.get("type") == "assistant" and isinstance(content, list):
  … 18 more lines in captures/34-timeline-src.txt
(exit 0 · 0.0 s)
```

รูปที่ 41: ซอร์สของ timeline.py ทั้งไฟล์

ส่วนแรกคือตัวกรอง. สคริปต์เก็บเฉพาะสองแบบ ไม่เก็บอย่างอื่นเลย.

```
KEEP_CMD = re.compile(r"herdr-room (in|out)|agent prompt wCC|pane
close|…")
KEEP_MSG = re.compile(r"^\[(wCC:p\d|right|left)")
```

`KEEP_CMD` จับคำสั่ง Bash ที่เกี่ยวกับการย้าย ส่วน `KEEP_MSG` จับข้อความที่ขึ้นต้นด้วย `[wCC:p`, `[right` หรือ `[left` คือข้อความที่ส่งมาจาก pane อื่น.

ส่วนที่สองคือเวลา. transcript เก็บเป็น UTC สคริปต์จึงแปลงด้วย `BKK = timezone(timedelta(hours=7))` ใน `when()` ให้เป็น Asia/Bangkok. นี่คือเหตุผลที่ capture 15 โชว์ 13:05:21Z แต่ในหนังสือเขียน 20:05:21.

ส่วนที่สามคือลูปหลัก. อ่านทีละบรรทัดเป็น JSON ถ้า `type` เป็น `assistant` ก็หา `tool_use` ที่ชื่อ `Bash` แล้วตัดคำสั่งด้วย `;`, `&&` และขึ้นบรรทัดใหม่ เก็บเฉพาะท่อนที่ตรง

`KEEP_CMD` พิมพ์เป็น `run`. ถ้าเป็นข้อความขาเข้า (`user` หรือ `queue-operation`) ก็เก็บบรรทัดที่ตรง `KEEP_MSG` พิมพ์เป็น `msg` และใช้เซต `seen` กันบรรทัดซ้ำ.

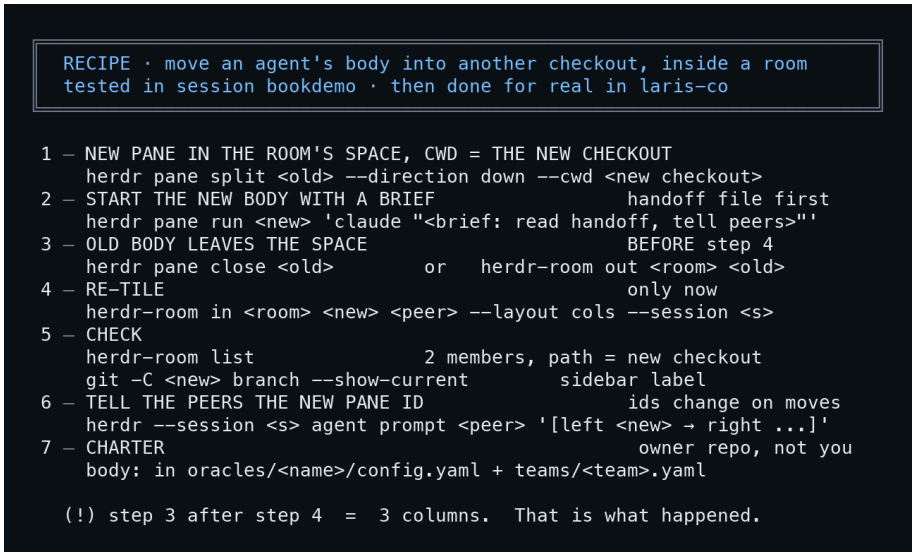
สคริปต์กรองอย่างเดียว ไม่เติมข้อความ (กรองแล้วตัดที่ 100 ตัวอักษรและกันบรรทัดซ้ำจึงบางบรรทัดหายไปได้). ใช้เป็น

`python3 timeline.py <session.jsonl> --since 20:05 --until 20:07`.

8.5 สูตรย้ายร่าง

ขั้นที่ 1 — เก็บเจ็ดขั้นไว้ในการ์ดใบเดียว

ทั้งเล่มย่อลงมาเป็นการ์ดนี้. ขั้น 1–2 คือบทที่ 4, ขั้น 3–4 คือบทที่ 5–6, ขั้น 5–6 คือบทที่ 7 และขั้น 7 คือสิ่งที่บทนี้เพิ่งเห็น. การ์ดบอกลำดับที่ห้ามสลับไว้ตรง ๆ.



รูปที่ 42: การ์ดสูตรเจ็ดขั้น ขั้น 3 ร่างเก่าออกก่อนขั้น 4 re-tile

จุดที่พลาดจริงคือขั้น 3 กับ 4. **ให้ร่างเก่าออกจาก space ก่อน แล้วค่อย `room in`**. คืนนั้น `room in` รันตอน 20:05:38 ขณะ p3 ยังเปิดอยู่ ห้องได้ 3 คอลัมน์ รอบ 20:05:48 ก็ยังได้ 3 ต้องรอรอบ 20:06:04 ถึงเหลือ 2. id ของ pane เปลี่ยนทุกครั้ง อย่าลืมบอกเพื่อนข้างขวาด้วย. คราวหน้าเปิดการ์ดนี้ข้างเทอร์มินัล แล้วเริ่มจากขั้นที่ 1.